Tugas Besar 1

DasarAgamaIslam

IF3070 - Dasar Intelegensi Artifisial

Disusun Oleh:

18224036	Muhammad Azka Firdaus
18224070	Almer Zain Farisseno
18224074	Fatin Sajjid Darwis

Dosen Pengampu :

Dr. Nur Ulfa Maulidevi, ST, M.Sc.

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

INSTITUT TEKNOLOGI BANDUNG

2025

DAFTAR ISI

DAFTAR ISI **1**
DESKRIPSI PERSOALAN **2**
PEMBAHASAN **4**
A. Objective Function 4
B. Implementasi Algoritma Local Search 5
1. Hill-Climbing Steepest Ascent 5
2. Hill-Climbing With Sideways Move 10
3. Random Restart Hill-Climbing 12
4. Stochastic Hill-Climbing 14
5. Simulated Annealing 16
6. Genetic Algorithm 21
HASIL & ANALISIS **26**
A. Hill-Climbing Steepest Ascent 27
B. Hill-Climbing With Sideways Move 32
C. Random Restart Hill-Climbing 37
D. Stochastic Hill-Climbing 41
E. Simulated Annealing 45
F. Genetic Algorithm 52
KESIMPULAN DAN SARAN **79**
A. Kesimpulan 79
B. Saran 81
REFERENSI **82**
SURAT PERNYATAAN PENGGUNAAN AI **83**

DESKRIPSI PERSOALAN

PT Owen Intim Line adalah perusahaan pelayaran yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan baru saja memperoleh kontrak pengiriman dari beberapa dealer kendaraan sekaligus. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman (Shipping Fee), dan deadline (ETA) yang berbeda-beda. Selama ini penempatan kendaraan pada geladak kapal dilakukan secara manual, sehingga ruang geladak sering tidak termanfaatkan dengan baik dan pemilihan kendaraan yang diangkut belum tentu menghasilkan keuntungan yang optimal.

Sebagai Forward Deployed Engineer, kami diminta membangun sistem pendukung keputusan yang menentukan kendaraan mana yang diangkut, serta posisi dan orientasi setiap kendaraan pada geladak. Tujuannya adalah memaksimalkan total Shipping Fee kendaraan yang diangkut. Karena yang dicari adalah konfigurasi akhir terbaik, bukan urutan langkah untuk mencapainya, persoalan ini cocok diselesaikan dengan algoritma local search.

Geladak kapal dan setiap kendaraan direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang dengan koordinat bilangan bulat. Kendaraan memiliki atribut id, dimensi (lebar dan panjang), orientasi (Horizontal atau Vertical), Shipping Fee, berat, dan ETA, sedangkan kapal memiliki dimensi geladak dan kapasitas angkut maksimum (MaxCapacity). Tidak semua kendaraan harus diangkut; kendaraan yang tidak dimuat berada sepenuhnya di luar kapal. Setiap solusi wajib memenuhi tiga batasan berikut:

1. Tidak saling bertumpuk, yaitu tidak ada sel geladak yang ditempati lebih dari satu kendaraan.
2. Berada di dalam batas geladak, yaitu seluruh sel yang ditempati kendaraan berada di dalam dimensi geladak.
3. Tidak melebihi kapasitas, yaitu total berat kendaraan yang diangkut tidak melebihi MaxCapacity kapal.

State awal dibangkitkan secara acak. Dari sebuah state, langkah neighbor (move) yang diperbolehkan pada setiap iterasi adalah:

1. menukar dua kendaraan, baik yang berada di dalam maupun di luar kapal;
2. memindahkan sebuah kendaraan ke koordinat yang berbeda;
3. mengubah orientasi sebuah kendaraan.

State akhir harus dapat menunjukkan keadaan semua kendaraan, baik yang berada di dalam maupun di luar kapal, keadaan geladak, dan nilai objective function state tersebut.

Persoalan ini diselesaikan dengan tiga kelompok algoritma local search, yaitu Hill-Climbing (varian Steepest Ascent, Sideways Move, Random Restart, dan Stochastic), Simulated Annealing, dan Genetic Algorithm. Setiap algoritma selain Genetic Algorithm dijalankan sebanyak tiga kali. Genetic Algorithm diuji dengan dua skenario: jumlah populasi tetap dengan tiga variasi banyak iterasi, dan banyak iterasi tetap dengan tiga variasi jumlah populasi, masing-masing tiga kali. Untuk setiap percobaan dicatat state awal dan akhir, nilai objective function akhir, grafik nilai objective function terhadap banyak iterasi, dan durasi pencarian, ditambah informasi khusus tiap algoritma seperti banyak iterasi, banyak restart, grafik $e^{\frac{\Delta E}{T}}$, dan frekuensi terjebak pada Simulated Annealing. Hasil tersebut kemudian dianalisis untuk membandingkan kualitas solusi, durasi, dan konsistensi setiap algoritma.

PEMBAHASAN

A. Objective Function

Masalah yang dibahas adalah pemilihan dan penataan kendaraan pada geladak kapal. Sebuah solusi (state) direpresentasikan sebagai daftar dengan satu entri untuk setiap kendaraan. Entri berisi koordinat dan orientasi kendaraan, yaitu (x, y, orientasi), atau None jika kendaraan tidak diangkut. Orientasi H menggunakan dimensi asli kendaraan, sedangkan orientasi V menukar lebar dan panjangnya.

Misalkan terdapat himpunan kendaraan V = {1, 2, ..., n}. Untuk kendaraan ke-i, definisikan:

- f_i : ShippingFee kendaraan ke-i.
- w_i : berat kendaraan ke-i.
- $z_i \in \{0, 1\}$: bernilai 1 jika kendaraan dimuat dan 0 jika tidak.

Setiap kendaraan yang dimuat juga memiliki posisi (x_i, y_i) dan orientasi $o_i \in \{H, V\}$. Kualitas solusi diukur dengan total ShippingFee kendaraan yang dimuat:

$$F(S) = \Sigma\,(f_i \times z_i)\ (untuk\ 1 = 1\ sampai\ n)$$

Fungsi objektif dimaksimalkan. Dengan demikian, solusi yang lebih baik adalah solusi valid yang menghasilkan total ShippingFee lebih besar. Posisi dan orientasi tidak menambah nilai secara langsung; keduanya menentukan apakah kendaraan dapat dimuat tanpa melanggar batasan.

Solusi harus memenuhi seluruh batasan berikut:

1. **Berada di dalam geladak.** Seluruh sel yang ditempati kendaraan harus berada di dalam ukuran kapal. Untuk kapal berukuran W × L, kendaraan dengan ukuran setelah rotasi w'_i × l'_i harus memenuhi $0 \le x_i$, $0 \le y_i$, $x_i + w'_i \le W$, dan $y_i + l'_i \le L$.

2. **Tidak saling bertumpuk.** Himpunan sel yang ditempati dua kendaraan yang berbeda tidak boleh beririsan.
3. **Kapasitas berat.** Total berat seluruh kendaraan yang dimuat tidak boleh melebihi kapasitas kapal C:

$$\Sigma\,(w_i \times z_i) \;\leq C\;(untuk\; i \;=\; 1\; sampai\; n)$$

Ketiga batasan tersebut diperiksa oleh state_is_fully_valid() melalui is_placement_valid(). Fungsi compute_fitness() mengembalikan 0 jika state tidak valid; jika valid, fungsi menjumlahkan ShippingFee kendaraan yang dimuat. Algoritma membangkitkan atau menerima hanya state yang valid, sehingga nilai 0 pada state tidak valid berfungsi sebagai pemeriksaan pengaman, bukan penalti yang menjadi bagian dari fungsi objektif.

Batas atas sederhana untuk fungsi objektif adalah jumlah ShippingFee seluruh kendaraan:

$$F_{atas} \;=\; \Sigma f_i\;(untuk\; i \;=\; 1\; sampai\; n)$$

Batas ini tercapai jika semua kendaraan dapat dimuat sekaligus dan seluruh batasan tetap terpenuhi. Jika nilai ini tercapai, tidak ada solusi yang dapat memiliki nilai objektif lebih tinggi. Namun, jika kombinasi ukuran, berat, dan posisi membuat sebagian kendaraan tidak dapat dimuat bersamaan, batas atas tersebut belum tentu dapat dicapai. Karena itu, berhenti pada suatu state atau menyamai batas atas hanya dapat disebut optimal global jika batas atas tersebut memang dapat dicapai.

B. Implementasi Algoritma Local Search

1. Hill-Climbing Steepest Ascent

Input	**Output**
vehicles, ship, initial_state, seed	best_state, best_val, initial_state

```
def steepest_hill_climbing(vehicles, ship, initial_state=None, seed=None):
    if seed is not None:
        random.seed(seed)

    if initial_state is None:
        initial_state = random_initial_state(vehicles, ship)

    state = initial_state[:]
    cur_val = compute_fitness(state, vehicles, ship)
    iteration = 0

    while True:
        neighbors = (
            move_swap(state, vehicles, ship)
            + move_relocate(state, vehicles, ship)
            + move_rotate(state, vehicles, ship)
        )

        if not neighbors:
            break

        # Hitung nilai fitness seluruh neighbor.
        scored_neighbors = [
            (neighbor, compute_fitness(neighbor, vehicles, ship))
            for neighbor in neighbors
        ]

        # Pilih neighbor dengan nilai fitness tertinggi.
        best_neighbor, best_neighbor_val = max(
            scored_neighbors,
            key=lambda item: item[1]
        )

        # Berhenti jika tidak ada neighbor yang lebih baik.
        if best_neighbor_val <= cur_val:
            break

        state = best_neighbor
        cur_val = best_neighbor_val
        iteration += 1

        if iteration % 10 == 0:
```

```
            print(
                f"\rIterasi {iteration:>7d} "
                f"cur={cur_val:.1f}",
                end="",
                flush=True
            )

        print()
        best_state = state[:]
        best_val = cur_val

        return best_state, best_val, initial_state
```

Fungsi steepest_hill_climbing menerima vehicles, ship, initial_state, dan seed. Jika seed diberikan, seed tersebut digunakan untuk mengatur proses acak. Jika initial_state tidak diberikan, fungsi membuat state awal menggunakan random_initial_state.

Pada setiap iterasi, fungsi membangkitkan seluruh neighbor dari aksi move_swap, move_relocate, dan move_rotate. Nilai fitness setiap neighbor kemudian dihitung. Fungsi memilih neighbor dengan fitness tertinggi dan berpindah ke neighbor tersebut hanya jika nilainya lebih besar dari fitness state saat ini.

Pencarian berhenti ketika tidak ada neighbor yang tersedia atau neighbor terbaik tidak menghasilkan peningkatan fitness. Fungsi kemudian mengembalikan state terbaik yang ditemukan, nilai fitness-nya, dan state awal. Karena algoritma ini mencari neighbor terbaik di setiap langkah, hasilnya adalah optimum lokal dan belum tentu optimum global.

Actions (Sama untuk semua varian hill-climbing)

a. move_swap

```
    def move_swap(state, vehicles, ship):
        n = len(vehicles)
        neighbors = []

        if n < 2:
            return neighbors

        for i in range(n):
```

```
            for j in range(i + 1, n):
                candidate = state[:]
                candidate[i], candidate[j] = candidate[j], candidate[i]

                ok_move = (
                    is_placement_valid(candidate, i, vehicles, ship)
                    and is_placement_valid(candidate, j, vehicles, ship)
                    and total_weight(candidate, vehicles) <= ship.max_cap
                )

                if ok_move and candidate != state:
                    neighbors.append(candidate)

        return neighbors
```

Fungsi move_swap menerima state, vehicles, dan ship, lalu menghasilkan daftar neighbor dari semua kemungkinan pertukaran posisi dua kendaraan. Setiap pasangan indeks dicoba satu kali. Kandidat dimasukkan ke daftar neighbor jika penempatan kendaraan tetap valid dan total bobot tidak melebihi kapasitas kapal. Jika jumlah kendaraan kurang dari dua, tidak ada pertukaran yang mungkin dilakukan sehingga fungsi mengembalikan daftar kosong.

b. move_relocate

```
    def move_relocate(state, vehicles, ship):
        neighbors = []

        for i, veh in enumerate(vehicles):
            for ori in ("H", "V"):
                cw, ch = (veh.w, veh.l) if ori == "H" else (veh.l, veh.w)

                if cw > ship.w or ch > ship.l:
                    continue

                for x in range(ship.w - cw + 1):
                    for y in range(ship.l - ch + 1):
                        candidate = state[:]
                        candidate[i] = (x, y, ori)

                        ok_move = (
                            is_placement_valid(candidate, i, vehicles, ship)
                            and total_weight(candidate, vehicles) <= ship.max_cap
```

```
                )

                if ok_move and candidate != state:
                    neighbors.append(candidate)

        return neighbors
```

Fungsi move_relocate mencoba memindahkan setiap kendaraan ke seluruh koordinat dan orientasi yang memungkinkan. Orientasi dipilih dengan mencoba 'H' dan 'V'; ukuran kendaraan disesuaikan berdasarkan orientasinya. Posisi kandidat hanya ditambahkan ke daftar neighbor jika berada di dalam batas kapal, penempatannya valid, dan total bobot tidak melebihi kapasitas kapal.

c. move_rotate

```
    def move_rotate(state, vehicles, ship):
        neighbors = []

        for i, placement in enumerate(state):
            if placement is None:
                continue

            x, y, ori = placement
            new_ori = "V" if ori == "H" else "H"

            candidate = state[:]
            candidate[i] = (x, y, new_ori)

            ok_move = (
                is_placement_valid(candidate, i, vehicles, ship)
                and total_weight(candidate, vehicles) <= ship.max_cap
            )

            if ok_move and candidate != state:
                neighbors.append(candidate)

        return neighbors
```

Fungsi move_rotate mencoba memutar setiap kendaraan yang sudah memiliki posisi. Kendaraan dengan orientasi 'H' dicoba diubah menjadi 'V', dan sebaliknya, tanpa mengubah koordinatnya. Kandidat ditambahkan ke daftar neighbor jika penempatan setelah rotasi tetap valid dan total bobot masih memenuhi kapasitas kapal.

2. Hill-Climbing With Sideways Move

Input	Output
vehicles, ship, initial_state, seed, max_sideways	best_state, best_val, initial_state

```
def sideways_hill_climbing(
    vehicles,
    ship,
    initial_state=None,
    seed=None,
    max_sideways=100
):
    if seed is not None:
        random.seed(seed)

    if initial_state is None:
        initial_state = random_initial_state(vehicles, ship)

    state = initial_state[:]
    cur_val = compute_fitness(state, vehicles, ship)

    best_state = state[:]
    best_val = cur_val

    sideways_count = 0
    iteration = 0

    while True:
        neighbors = (
            move_swap(state, vehicles, ship)
            + move_relocate(state, vehicles, ship)
            + move_rotate(state, vehicles, ship)
        )
```

```
if not neighbors:
    break

scored_neighbors = [
    (neighbor, compute_fitness(neighbor, vehicles, ship))
    for neighbor in neighbors
]

# Pilih neighbor dengan fitness tertinggi.
best_neighbor, best_neighbor_val = max(
    scored_neighbors,
    key=lambda item: item[1]
)

if best_neighbor_val > cur_val:
    # Langkah yang meningkatkan fitness.
    state = best_neighbor
    cur_val = best_neighbor_val
    sideways_count = 0

    if cur_val > best_val:
        best_state = state[:]
        best_val = cur_val

elif best_neighbor_val == cur_val and sideways_count < max_sideways:
    # Langkah dengan fitness sama, selama batas belum tercapai.
    state = best_neighbor
    sideways_count += 1

else:
    # Berhenti jika neighbor terbaik lebih buruk,
    # atau batas langkah sideways sudah tercapai.
    break

iteration += 1

if iteration % 10 == 0:
    print(
        f"\rIterasi {iteration:>7d} "
        f"cur={cur_val:.1f} "
        f"best={best_val:.1f} "
        f"sideways={sideways_count}",
        end="",
        flush=True
```

```
            )

        print()
        return best_state, best_val, initial_state
```

Fungsi sideways_hill_climbing menerima vehicles, ship, initial_state, seed, dan max_sideways. Parameter max_sideways menentukan jumlah maksimum langkah berturut-turut yang boleh diambil ketika fitness tidak berubah. Jika initial_state tidak diberikan, fungsi membuat state awal menggunakan random_initial_state.

Pada setiap iterasi, fungsi membangkitkan seluruh neighbor menggunakan aksi move_swap, move_relocate, dan move_rotate, lalu menghitung fitness masing-masing. Fungsi memilih neighbor dengan fitness tertinggi. Jika fitness neighbor lebih tinggi, algoritma berpindah ke neighbor tersebut dan mengatur ulang penghitung sideways menjadi nol. Jika fitness-nya sama, algoritma boleh berpindah selama jumlah langkah sideways belum mencapai max_sideways.

Pencarian berhenti jika tidak ada neighbor, neighbor terbaik memiliki fitness lebih rendah, atau batas langkah sideways tercapai. Fungsi mengembalikan state dengan fitness tertinggi yang pernah ditemukan, nilai fitness-nya, dan state awal. Sideways move membantu algoritma bergerak melewati plateau, tetapi tidak menjamin menemukan optimum global.

3. Random Restart Hill-Climbing

Input	Output
vehicles, ship, initial_state, seed, max_restarts	best_state, best_val, initial_state

```
    def random_restart_hill_climbing(
        vehicles,
        ship,
        initial_state=None,
        seed=None,
```

```
    max_restarts=20
):
    if seed is not None:
        random.seed(seed)

    # Simpan state awal pertama untuk dikembalikan.
    if initial_state is None:
        initial_state = random_initial_state(vehicles, ship)
    first_initial_state = initial_state[:]

    best_state = None
    best_val = float("-inf")

    # max_restarts adalah jumlah restart tambahan.
    for restart in range(max_restarts + 1):
        if restart == 0:
            state = first_initial_state[:]
        else:
            state = random_initial_state(vehicles, ship)

        cur_val = compute_fitness(state, vehicles, ship)

        # Steepest hill climbing untuk satu initial state.
        while True:
            neighbors = (
                move_swap(state, vehicles, ship)
                + move_relocate(state, vehicles, ship)
                + move_rotate(state, vehicles, ship)
            )

            if not neighbors:
                break

            scored_neighbors = [
                (neighbor, compute_fitness(neighbor, vehicles, ship))
                for neighbor in neighbors
            ]

            # Pilih neighbor dengan fitness tertinggi.
            best_neighbor, best_neighbor_val = max(
                scored_neighbors,
                key=lambda item: item[1]
            )

            # Berhenti jika tidak ada peningkatan.
```

```
            if best_neighbor_val <= cur_val:
                break

            state = best_neighbor
            cur_val = best_neighbor_val

        # Simpan hasil terbaik dari seluruh restart.
        if cur_val > best_val:
            best_state = state[:]
            best_val = cur_val

        print(
            f"Restart {restart + 1}/{max_restarts + 1}: "
            f"fitness={cur_val:.1f}, best={best_val:.1f}"
        )

    return best_state, best_val, first_initial_state
```

Fungsi random_restart_hill_climbing menerima state awal opsional, seed, dan jumlah restart tambahan. Jika initial_state tidak diberikan, fungsi membuat state awal secara acak. State tersebut digunakan untuk pencarian pertama; pencarian berikutnya dimulai dari state acak baru.

Untuk setiap restart, fungsi menjalankan steepest hill climbing: membangkitkan semua neighbor, memilih neighbor dengan fitness tertinggi, lalu berpindah ke sana jika fitness meningkat. Pencarian lokal berhenti ketika tidak ada neighbor yang lebih baik. Setelah itu, hasil restart tersebut dibandingkan dengan hasil terbaik sejauh ini.

max_restarts=20 berarti algoritma melakukan 21 pencarian lokal: satu pencarian awal dan 20 restart tambahan. Fungsi mengembalikan state terbaik dari semua pencarian, nilai fitness-nya, dan state awal pertama.

4. Stochastic Hill-Climbing

Input	**Output**
vehicles, ship, initial_state, seed, max_iterations	best_state, best_val, initial_state

```
def stochastic_hill_climbing(
    vehicles,
    ship,
    initial_state=None,
    seed=None,
    max_iterations=10000
):
    if seed is not None:
        random.seed(seed)

    if initial_state is None:
        initial_state = random_initial_state(vehicles, ship)

    state = initial_state[:]
    cur_val = compute_fitness(state, vehicles, ship)

    best_state = state[:]
    best_val = cur_val

    for iteration in range(1, max_iterations + 1):
        neighbors = (
            move_swap(state, vehicles, ship)
            + move_relocate(state, vehicles, ship)
            + move_rotate(state, vehicles, ship)
        )

        # Simpan neighbor yang meningkatkan fitness.
        better_neighbors = []
        for neighbor in neighbors:
            neighbor_val = compute_fitness(neighbor, vehicles, ship)
            if neighbor_val > cur_val:
                better_neighbors.append((neighbor, neighbor_val))

        # Berhenti jika tidak ada neighbor yang lebih baik.
        if not better_neighbors:
            break

        # Pilih secara acak salah satu neighbor yang lebih baik.
        state, cur_val = random.choice(better_neighbors)

        if cur_val > best_val:
            best_state = state[:]
            best_val = cur_val
```

```
            if iteration % 10 == 0:
                print(
                    f"\rIterasi {iteration:>7d} "
                    f"cur={cur_val:.1f} "
                    f"best={best_val:.1f}",
                    end="",
                    flush=True
                )

        print()
        return best_state, best_val, initial_state
```

Fungsi stochastic_hill_climbing menerima vehicles, ship, initial_state, seed, dan max_iterations. Jika initial_state tidak diberikan, fungsi membuat state awal menggunakan random_initial_state.

Pada setiap iterasi, fungsi membangkitkan neighbor dari aksi move_swap, move_relocate, dan move_rotate. Fitness setiap neighbor dihitung, lalu neighbor yang fitness-nya lebih tinggi dari state saat ini dimasukkan ke daftar better_neighbors. Algoritma memilih satu neighbor secara acak dari daftar tersebut dan berpindah ke sana.

Pencarian berhenti jika tidak ada neighbor yang meningkatkan fitness atau jumlah iterasi mencapai max_iterations. Fungsi mengembalikan state terbaik yang ditemukan, nilai fitness-nya, dan state awal. Karena pemilihan neighbor dilakukan secara acak, jalur pencarian dapat berbeda pada setiap eksekusi jika seed tidak ditetapkan.

5. Simulated Annealing

Input	Output
t, T0, alpha, Tmin	best_state, best_val, initial_state

```
        def schedule(t, T0, alpha=0.995, T_min=0.001):
```

```
            T = T0 * alpha ** (t - 1)
            return T if T >= T_min else 0.0

        if seed is not None:
            random.seed(seed)

            if initial_state is None:
                initial_state = random_initial_state(vehicles, ship)

            state   = initial_state[:]
            cur_val = compute_fitness(state, vehicles, ship)
            best_state = state[:]
            best_val   = cur_val

            move_funcs = [move_swap, move_relocate, move_rotate]
            if record is not None:
                record.update(iteration=[0], value=[cur_val], best_so_far=[best_val],
                              acceptance_probability=[1.0], temperature=[T0])

            for t in itertools.count(1):
                T = schedule(t, T0)
                if T == 0:
                    break

                new_state = random.choice(move_funcs)(state, vehicles, ship)
                new_val   = compute_fitness(new_state, vehicles, ship)
                delta     = new_val - cur_val

                if delta >= 0 or random.random() < math.exp(delta / T):
                    state, cur_val = new_state, new_val

                if cur_val > best_val:
                    best_val   = cur_val
                    best_state = state[:]

                if record is not None:
                    record["iteration"].append(t)
                    record["value"].append(cur_val)
                    record["best_so_far"].append(best_val)
                    record["acceptance_probability"].append(1.0 if delta >= 0 else math.exp(delta /
T))
                    record["temperature"].append(T)

                if (t + 1) % 10 == 0:
                            print(f"\r    Iterasi  {t+1:>7d}  "  f"T={T:.2f}    "  f"cur={cur_val:.1f}
```

```
best={best_val:.1f}   " , end="", flush=True)

            print()
            if record is not None:
                record["final_current_state"] = state[:]
            return best_state, best_val, initial_state
```

Fungsi simulated_annealing dimulai dengan menerima beberapa parametar input, yaitu *vehicles, ship*, *initial_state, T0*, dan *seed.* Fungsi kemudian melakukan validasi ketersediaan *seed* dan *initial_state* yang diberikan. Selanjutnya, fungsi meng-*assign* paramater input yang telah diterima ke variabel awal seperti bentuk state awal (state) dan nilai state awal (cur_val). Kemudian, sebuah record juga akan dibuat untuk menyimpan *history* state-state setiap iterasi.

Pada iterasi, t diberikan nilai 1 hingga tak hingga, hal ini memastikan bahwa algoritma terus mencari hingga global optimum ditemukan. Iterasi hanya akan berhenti ketika T mencapai 0, yaitu ketika T0 * alpha ** (t - 1) kurang dari T_min. Setiap iterasi, algoritma akan memilih salah satu aksi (move_swap, move_relocate, move_rotate) secara acak dan menghitung kembali nilai yang ada. Jika nilai sekarang lebih besar dibandingkan nilai sebelumnya, maka state nilai sekarang akan dipakai, tetapi jika nilai sekarang kurang dari nilai sebelumnya, maka akan state nilai sekarang akan memiliki probabilitas $e^{\Delta E/T}$ dimana ΔE adalah nilai sekarang dikurangi nilai sebelumnya.

Terakhir fungsi akan mengembalikan nilai terbaik, state_terbaik, dan state awal. Variabel yang dikembalikan tersebut akan digunakan untuk visualisasi dan bahan analisis algoritma tersebut. Begitu juga, record yang ada akan disimpan.

Actions

a. move_swap

```
def move_swap(state, vehicles, ship):
    n = len(vehicles)
    if n < 2:
```

```
    return state[:]
i, j = random.sample(range(n), 2)
new_state = state[:]
new_state[i], new_state[j] = state[j], state[i]
ok_move = (is_placement_valid(new_state, i, vehicles, ship) and
     is_placement_valid(new_state, j, vehicles, ship) and
     total_weight(new_state, vehicles) <= ship.max_cap)
return new_state if ok_move else state[:]
```

Fungsi move_swap diawali dengan menerima input berupa state, vehicles, dan ship. Pertama, variabel n akan diinisialisasikan untuk menyimpan total kendaraan. Kalau jumlah kendaraan ternyata kurang dari 2, fungsi langsung mengembalikan salinan state awal karena penukaran posisi tidak memungkinkan untuk dilakukan.

Jika kriteria jumlah terpenuhi, fungsi memilih dua indeks kendaraan secara acak, yaitu i dan j. Proses penukaran dilakukan pada salinan state baru (new_state) dengan menukar posisi kendaraan di kedua indeks tersebut. Setelah itu, fungsi akan mengevaluasi kelayakan kondisi lewat variabel ok_move untuk memastikan bahwa posisi baru kendaraan i dan j tetap *valid* secara aturan penempatan, serta total bobot seluruh kendaraan tidak melebihi kapasitas maksimal kapal (ship.max_cap). Jika seluruh kondisi ini terpenuhi, new_state yang akan dipakai, jika tidak, fungsi membatalkan perubahan dan mengembalikan salinan state semula.

b. move_relocate

```
def move_relocate(state, vehicles, ship):
    n = len(vehicles)
    i = random.randint(0, n - 1)
    veh = vehicles[i]
    new_state = state[:]

    ori = random.choice(['H', 'V'])
    cw, ch = (veh.w, veh.l) if ori == 'H' else (veh.l, veh.w)
    if cw > ship.w or ch > ship.l:
        new_state[i] = None
        return new_state

    x = random.randint(0, ship.w - cw)
    y = random.randint(0, ship.l - ch)
```

```
        cand = new_state[:]
        cand[i] = (x, y, ori)
        if (is_placement_valid(cand, i, vehicles, ship) and
                total_weight(cand, vehicles) <= ship.max_cap):
            new_state[i] = (x, y, ori)
        else:
            new_state[i] = None
        return new_state
```

Fungsi move_relocate diawali dengan menerima input berupa state, vehicles, dan ship. Pertama, variabel n akan diinisialisasikan untuk menyimpan total kendaraan. Fungsi kemudian memilih satu kendaraan secara acak berdasarkan indeks i, mengambil data kendaraan tersebut ke variabel veh, dan membuat salinan state awal ke new_state.

Selanjutnya, orientasi kendaraan dipilih secara acak antara horizontal ('H') atau vertikal ('V') melalui variabel ori. Berdasarkan orientasi tersebut, dimensi kendaraan (panjang dan lebar) akan disesuaikan ke variabel cw dan ch. Jika ukuran kendaraan ternyata melebihi dimensi kapal (ship.w atau ship.l), maka posisi kendaraan pada indeks i di new_state akan diubah menjadi None dan fungsi langsung mengembalikan new_state.

Jika ukurannya muat, fungsi akan menentukan koordinat posisi baru (x, y) secara acak di dalam area kapal. Salinan state kandidat (cand) dibuat untuk menampung posisi dan orientasi baru pada indeks i. Setelah itu, fungsi akan mengevaluasi kelayakan posisi baru tersebut untuk memastikan penempatan kendaraan valid serta total bobot tidak melebihi kapasitas maksimal kapal (ship.max_cap). Jika kondisi terpenuhi, new_state akan diperbarui dengan koordinat dan orientasi baru tersebut, sedangkan jika tidak valid, nilai pada indeks i akan diubah menjadi None sebelum new_state dikembalikan.

c. move_rotate

```
    def move_rotate(state, vehicles, ship):
        inside = [i for i, e in enumerate(state) if e is not None]
        if not inside:
            return state[:]
        i = random.choice(inside)
        x, y, ori = state[i]
```

```
        cand = state[:]
        cand[i] = (x, y, 'V' if ori == 'H' else 'H')
        if (is_placement_valid(cand, i, vehicles, ship) and
                total_weight(cand, vehicles) <= ship.max_cap):
            return cand
        return state[:]
```

Fungsi move_rotate diawali dengan menerima input berupa state, vehicles, dan ship. Pertama, variabel inside akan diinisialisasikan untuk mengumpulkan seluruh indeks kendaraan yang sudah memiliki posisi di dalam kapal atau nilainya bukan None. Jika daftar inside ternyata kosong, fungsi langsung mengembalikan salinan state awal karena tidak ada kendaraan yang bisa diputar.

Jika terdapat kendaraan di dalam kapal, fungsi akan memilih satu indeks i secara acak dari daftar inside tersebut. Koordinat awal (x, y) serta orientasi saat ini (ori) dari kendaraan pada indeks i kemudian diambil dari state. Selanjutnya, fungsi membuat salinan state baru (cand) dan mengubah orientasi kendaraan pada indeks i menjadi kebalikannya, yaitu dari 'H' ke 'V' atau sebaliknya. Setelah itu, fungsi akan mengevaluasi kelayakan rotasi tersebut untuk memastikan bahwa posisi dan orientasi baru kendaraan tetap valid secara aturan penempatan, serta total bobot seluruh kendaraan tidak melebihi kapasitas maksimal kapal (ship.max_cap). Jika kondisi terpenuhi, cand yang berisi state baru akan dikembalikan, namun jika tidak valid, fungsi akan mengembalikan salinan state semula.

6. Genetic Algorithm

Input	**Output**
vehicles, ship, population_size, max_iterations, seed, mutation_rate, mutation_out_prob, elitism, use_repair	GAResult (best_state, best_val, initial_state, history, ...)

```
        def genetic_algorithm(vehicles, ship, population_size=50, max_iterations=100, seed=None,
                              mutation_rate=0.1, mutation_out_prob=0.2, elitism=0, use_repair=False):
            prob, rng = Problem(vehicles, ship), np.random.default_rng(seed)
            t0 = time.perf_counter()

            population = [random_state(prob, rng) for _ in range(population_size)]
            fit = np.array([fitness(prob, s) for s in population])
            initial_state, initial_value = population[int(fit.argmax())], float(fit.max())
            history = {"iteration": [0], "max": [float(fit.max())], "mean": [float(fit.mean())]}

            gen = 0
            while gen < max_iterations and fit.max() < prob.max_fitness:
                gen += 1
                new_population = [population[i] for i in np.argsort(-fit)[:elitism]]
                while len(new_population) < population_size:
                    x = random_selection(population, fit, rng)
                    y = random_selection(population, fit, rng)
                    child = reproduce(x, y, rng)
                    if rng.random() < mutation_rate:
                        child = mutate(prob, child, rng, mutation_out_prob)
                    if use_repair:
                        child = repair(prob, child)
                    new_population.append(child)
                population = new_population
                fit = np.array([fitness(prob, s) for s in population])
                history["iteration"].append(gen)
                history["max"].append(float(fit.max()))
                history["mean"].append(float(fit.mean()))

            params = dict(population_size=population_size, max_iterations=max_iterations, seed=seed,
                          mutation_rate=mutation_rate, elitism=elitism, use_repair=use_repair)
            return GAResult(params, list(initial_state), list(population[int(fit.argmax())]), initial_value,
                            float(fit.max()), max(history["max"]), gen, time.perf_counter() - t0, history)
```

Fungsi genetic_algorithm menerima beberapa parameter, yaitu vehicles, ship, population_size, max_iterations, seed, mutation_rate, mutation_out_prob, elitism, dan use_repair. Pada tahap awal, objek Problem dibentuk untuk mengonversi data geladak dan kendaraan, serta generator angka acak NumPy diinisialisasi berdasarkan seed. Populasi awal

berukuran population_size dibangkitkan secara acak menggunakan fungsi random_state. Kebugaran (fitness) masing-masing individu dievaluasi, dan individu dengan nilai fitness tertinggi pada generasi awal dicatat sebagai initial_state dan initial_value.

Pada setiap generasi (iterasi), algoritma mempertahankan sejumlah individu terbaik jika parameter elitism diaktifkan. Sisa populasi baru dibentuk melalui seleksi dua induk menggunakan metode roulette-wheel (random_selection), rekombinasi satu titik potong (reproduce), serta peluang mutasi acak (mutate) sebesar mutation_rate. Jika fitur use_repair aktif, individu anak diperbaiki agar memenuhi batasan penataan. Proses evolusi ini terus berulang hingga generasi mencapai max_iterations atau ketika nilai fitness maksimum telah menyamai batas atas teoritis max_fitness. Fungsi mengembalikan objek GAResult yang memuat parameter eksekusi, state awal, state terbaik akhir, nilai objective function, rekam jejak konvergensi (history nilai maksimum dan rata-rata), durasi waktu pencarian, serta jumlah generasi yang diselesaikan.

Actions

a. random_selection

```
def random_selection(population, fit, rng):
    if fit.sum() <= 0:
        return population[int(rng.integers(len(population)))]
    return population[int(rng.choice(len(population), p=fit / fit.sum()))]
```

Fungsi random_selection menerima populasi individu, larik nilai fitness masing-masing individu (fit), dan generator acak rng. Pemilihan induk dilakukan menggunakan mekanisme seleksi proporsional kebugaran (roulette wheel selection), di mana probabilitas individu terpilih berbanding lurus dengan nilai fitness-nya relatif terhadap total fitness populasi (fit / fit.sum()). Jika seluruh individu dalam populasi memiliki total fitness bernilai nol atau tidak valid, fungsi akan memilih satu individu secara acak seragam.

b. reproduce

```
def reproduce(x, y, rng):
    c = int(rng.integers(1, len(x))) if len(x) > 1 else len(x)
    return x[:c] + y[c:]
```

Fungsi reproduce melakukan operasi pindah silang satu titik (single-point crossover) terhadap dua individu induk, yaitu x dan y. Titik potong c dipilih secara acak di antara rentang indeks kromosom. Kromosom anak (child) dibentuk dengan menggabungkan segmen gen dari induk x dari awal hingga indeks c - 1, dan menyambungkannya dengan segmen gen dari induk y mulai dari indeks c hingga gen terakhir

c. mutate

```
        def mutate(prob, child, rng, out_prob=0.2):
            child = child[:]
            i = int(rng.integers(prob.n))
            gene = None
            if rng.random() >= out_prob:
                for ori in ((H, V) if rng.random() < 0.5 else (V, H)):
                    dx, dy = prob.size(i, ori)
                    if dx <= prob.W and dy <= prob.L:
                        gene = (int(rng.integers(0, prob.W - dx + 1)), int(rng.integers(0, prob.L - dy
+ 1)), ori)
                        break
            child[i] = gene
            return child
```

Fungsi mutate menerima data persoalan prob, kromosom anak child, generator rng, serta parameter probabilitas peletakan di luar kapal out_prob. Satu gen kendaraan ke-i dipilih secara acak untuk dimutasi. Dengan probabilitas out_prob, gen tersebut dapat diubah menjadi None (dikeluarkan dari kapal). Sebaliknya, dengan probabilitas 1 - out_prob, algoritma memilih orientasi acak ('H' atau 'V') serta menentukan posisi koordinat acak (x, y) baru yang berada di dalam dimensi geladak kapal. Nilai gen pada indeks i kemudian diperbarui dan salinan kromosom hasil mutasi dikembalikan.

d. repair

```
        def repair(prob, state):
            state = state[:]
            occupied = np.zeros((prob.L, prob.W), dtype=bool)
            weight = 0.0
            density = prob.fee / np.array([w * l for w, l in zip(prob.vw, prob.vl)])
              for i in sorted((k for k, g in enumerate(state) if g is not None), key=lambda k:
-density[k]):
                x, y, ori = state[i]
```

```
                state[i] = None
                if weight + prob.weight[i] > prob.capacity:
                    continue
                candidates = []
                for rank, o in enumerate((ori, V if ori == H else H)):
                    dx, dy = prob.size(i, o)
                    if dx > prob.W or dy > prob.L:
                        continue
                    sat = np.pad(occupied.cumsum(0).cumsum(1), ((1, 0), (1, 0)))
                    free = (sat[dy:, dx:] - sat[:-dy, dx:] - sat[dy:, :-dx] + sat[:-dy, :-dx]) == 0
                            candidates += [(abs(cx - x) + abs(cy - y), rank, cy, cx, o) for cy, cx in
zip(*np.nonzero(free))]
                if candidates:
                    _, _, cy, cx, o = min(candidates)
                    cx, cy = int(cx), int(cy)
                    dx, dy = prob.size(i, o)
                    occupied[cy:cy + dy, cx:cx + dx] = True
                    weight += prob.weight[i]
                    state[i] = (cx, cy, o)
            return state
```

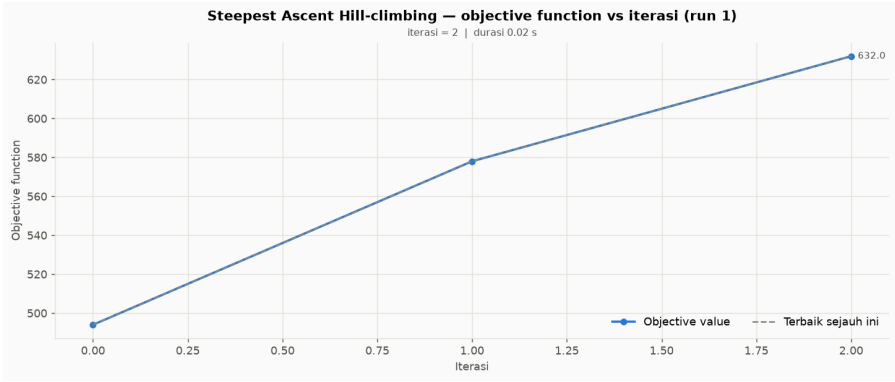
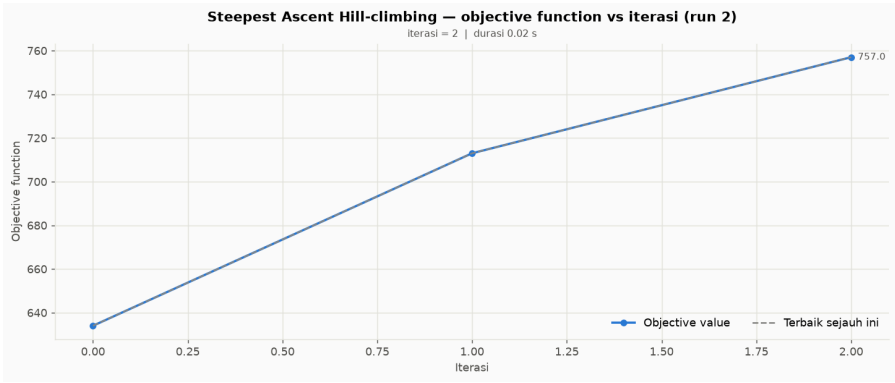
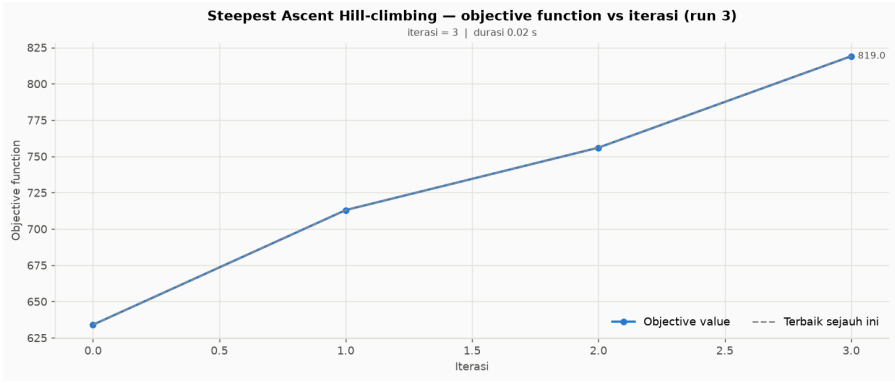
Fungsi repair berfungsi sebagai operator heuristik reparasi untuk memperbaiki susunan kendaraan pada suatu state yang mengalami pertumpukan (overlap) atau kelebihan kapasitas akibat operasi crossover maupun mutasi. Kendaraan diurutkan berdasarkan densitas nilai keuntungan terhadap luas area secara menurun (value density). Setiap kendaraan kemudian ditempatkan kembali pada posisi bebas terdekat menggunakan evaluasi matriks area terintegrasi (Summed-Area Table / SAT) tanpa melanggar batasan kapasitas bobot kapal. Jika tidak ditemukan ruang yang valid atau bobot telah melebihi batas, penempatan kendaraan tersebut diubah menjadi None.

HASIL & ANALISIS

Dalam eksperimen, kami menetapkan agar *state* awal untuk setiap percobaan merupakan *state* awal yang di-*generate* berbeda. Hal ini dikarenakan dengan adanya perubahan *state* awal, analisis performa untuk setiap algoritma akan lebih mudah dn lebih jelas untuk dilakukan. Hal ini juga akan menjadi acuan seberapa efektif algoritma *Local Search* yang dijalankan untuk memberikan solusi. Untuk setiap algoritma, kami menggunakan kapal dengan ukuran 10 x 10 dan kendaraan sebagai berikut:

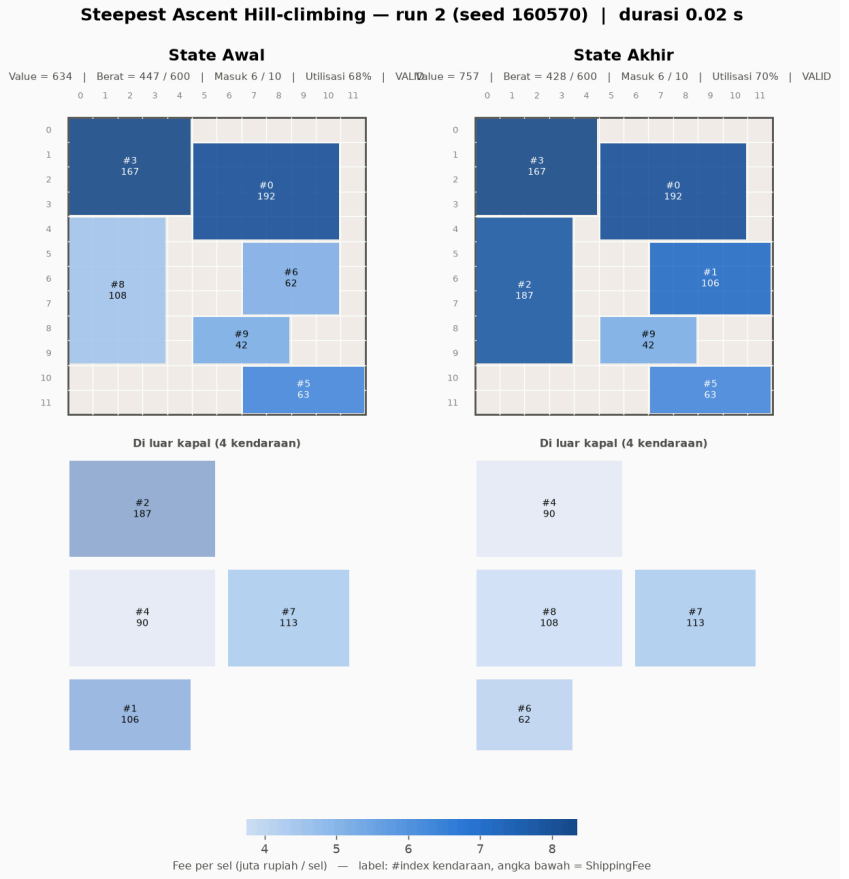
ID Kendaraan	**Width**	**Length**	**ShippingFee**	**Weight**
Owen-01	3	5	192.0	136.0
Avanza-02	2	4	106.0	58.0
Brio-03	3	5	187.0	88.0
Xenia-04	3	4	167.0	68.0
Ertiga-05	3	5	90.0	113.0
Fortuner-06	1	4	63.0	42.0
Pajero-07	2	3	62.0	54.0
Innova-08	3	4	113.0	72.0
Rush-09	3	5	108.0	111.0
Terios-10	1	3	42.0	36.0

A. Hill-Climbing Steepest Ascent

Percobaan	Grafik Nilai Objective Function terhadap Banyak Iterasi
1	
2	
3	

Percobaan	Perbandingan State

1	 Steepest Ascent Hill-climbing — run 1 (seed 160569) | durasi 0.02 s State Awal Value = 494 | Berat = 373 / 600 | Masuk 6 / 10 | Utilisasi 62% | VALID State Akhir Value = 632 | Berat = 394 / 600 | Masuk 6 / 10 | Utilisasi 62% | VALID #6 62 #7 113 #9 42 #8 108 #5 63 #1 106 #3 167 #0 192 Di luar kapal (4 kendaraan) #0 192 #2 187 #4 90 #3 167 #2 187 #4 90 #8 108 #7 113 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

2	 Steepest Ascent Hill-climbing — run 2 (seed 160570) | durasi 0.02 s State Awal Value = 634 | Berat = 447 / 600 | Masuk 6 / 10 | Utilisasi 68% | VALID State Akhir Value = 757 | Berat = 428 / 600 | Masuk 6 / 10 | Utilisasi 70% | VALID #3 167 #0 192 #8 108 #6 62 #9 42 #5 63 #2 187 #1 106 Di luar kapal (4 kendaraan) #4 90 #7 113 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

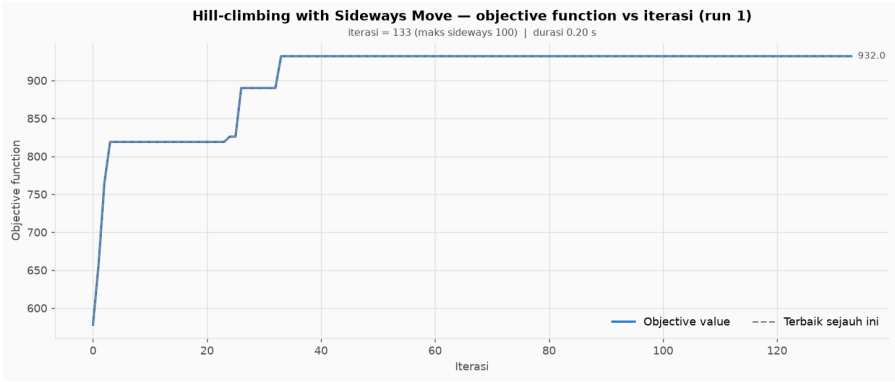
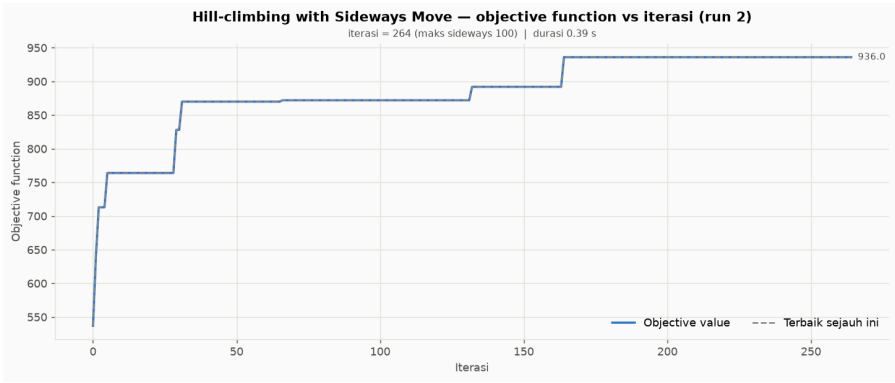
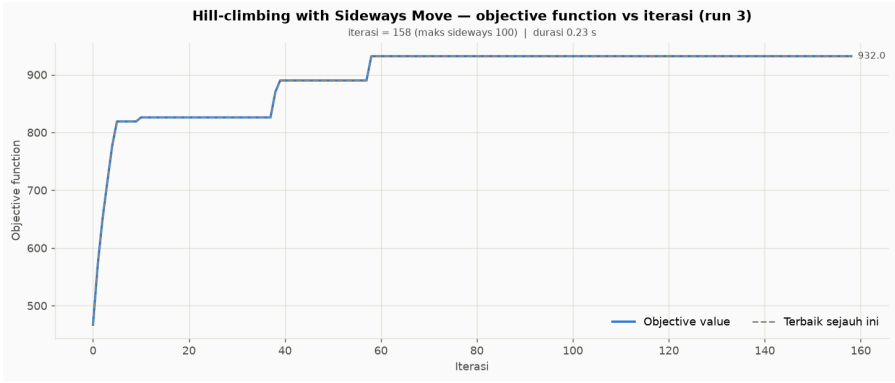
3	

Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.02	0.02	0.02
Jumlah Iterasi	2	2	3
Initial State	494	634	634
Final State	632	757	819

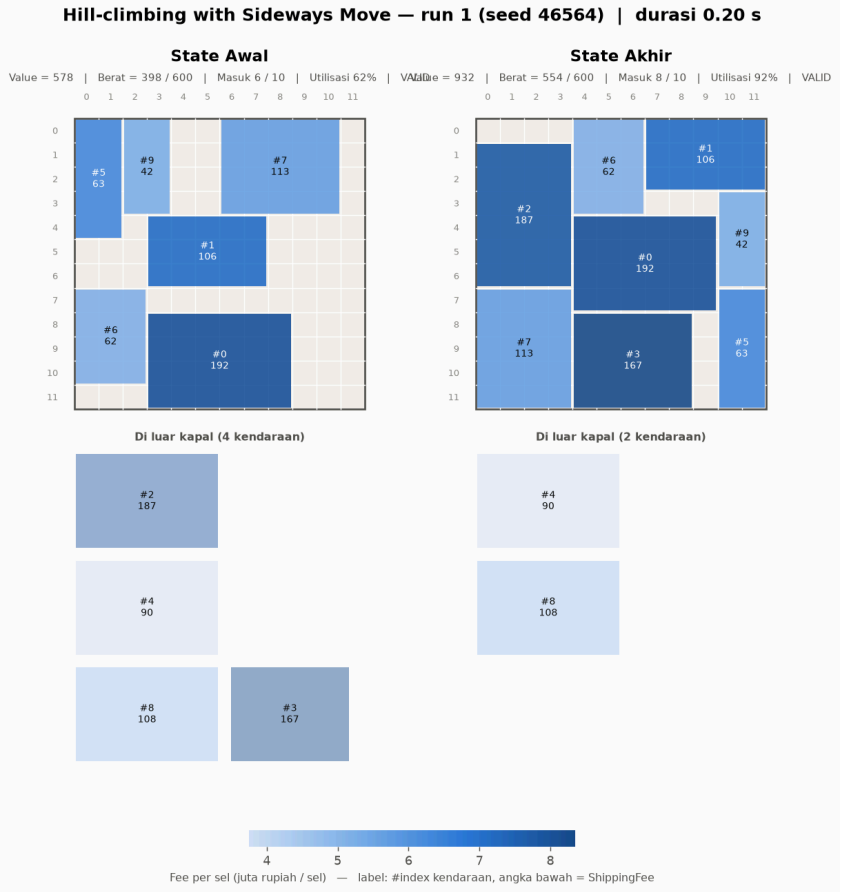
Pada ketiga percobaan, Steepest Ascent Hill-Climbing hanya membutuhkan 2–3 iterasi sebelum berhenti. Nilai objective function naik dari 494, 634, dan 634 menjadi 632, 757, dan 819, atau meningkat sebesar 138, 123, dan 185. Pada setiap iterasi algoritma memilih neighbor terbaik dari seluruh successor, sehingga kenaikan per iterasi besar, tetapi algoritma langsung berhenti begitu tidak ada neighbor yang nilainya lebih tinggi. Hal ini terlihat pada grafik yang hanya berisi beberapa titik dengan kenaikan tajam.

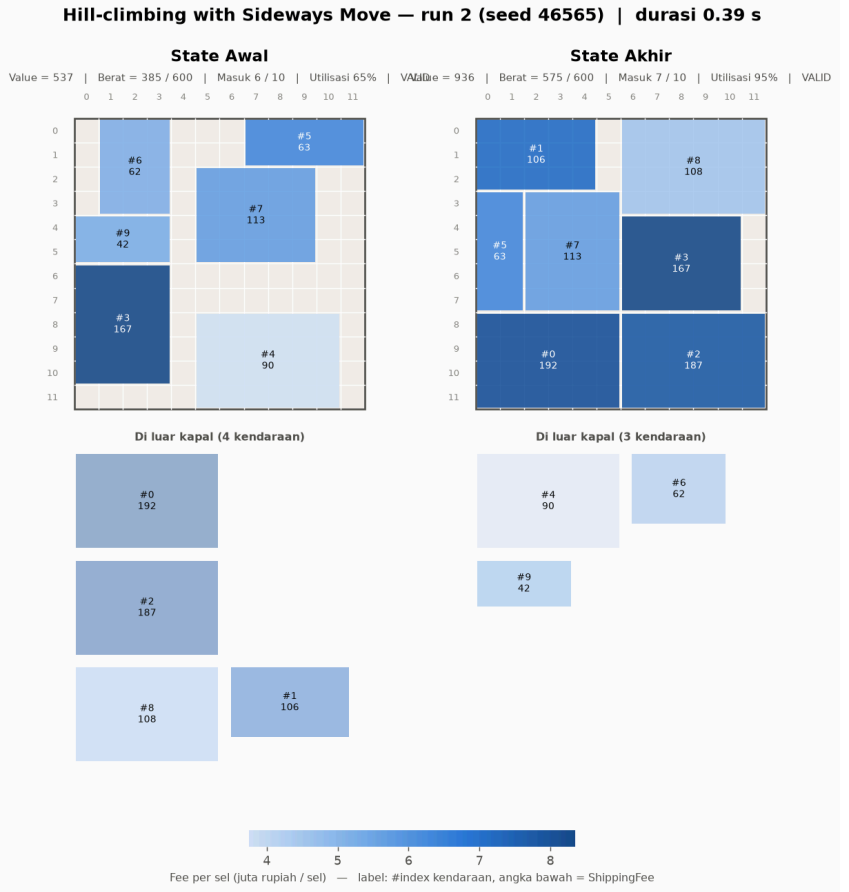
Nilai akhir ketiga percobaan masih cukup jauh dari optimum global 936, sehingga ketiganya berhenti di optimum lokal. Percobaan 2 dan 3 berangkat dari nilai awal yang sama (634), tetapi berakhir di nilai yang berbeda (757 dan 819). Artinya hasil Steepest Ascent sangat ditentukan oleh susunan state awal, bukan hanya nilai awalnya. Durasi sangat singkat (0,02 s) karena jumlah iterasinya sedikit.

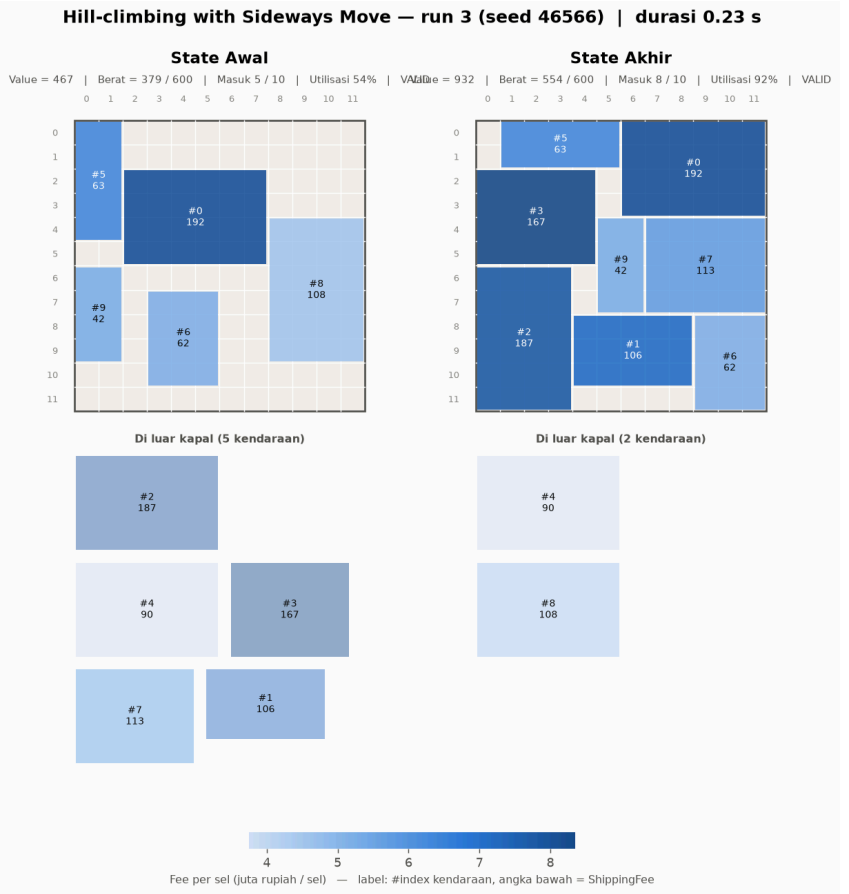
B. Hill-Climbing With Sideways Move

Percobaan	Grafik Nilai Objective Function terhadap Banyak Iterasi
1	
2	
3	

Percobaan	Perbandingan State

1	 Hill-climbing with Sideways Move — run 1 (seed 46564) | durasi 0.20 s State Awal Value = 578 | Berat = 398 / 600 | Masuk 6 / 10 | Utilisasi 62% | VALID #5 63 #9 42 #7 113 #1 106 #6 62 #0 192 State Akhir Value = 932 | Berat = 554 / 600 | Masuk 8 / 10 | Utilisasi 92% | VALID #6 62 #1 106 #2 187 #9 42 #0 192 #7 113 #3 167 #5 63 Di luar kapal (4 kendaraan) #2 187 #4 90 #8 108 #3 167 Di luar kapal (2 kendaraan) #4 90 #8 108 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

2	 Hill-climbing with Sideways Move — run 2 (seed 46565) | durasi 0.39 s State Awal Value = 537 | Berat = 385 / 600 | Masuk 6 / 10 | Utilisasi 65% | VALID State Akhir Value = 936 | Berat = 575 / 600 | Masuk 7 / 10 | Utilisasi 95% | VALID #5 63 #6 62 #7 113 #9 42 #3 167 #4 90 #1 106 #8 108 #0 192 #2 187 Di luar kapal (4 kendaraan) Di luar kapal (3 kendaraan) 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

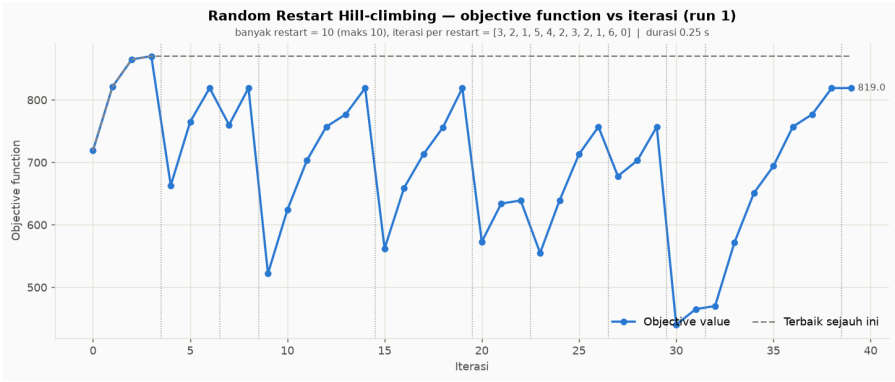
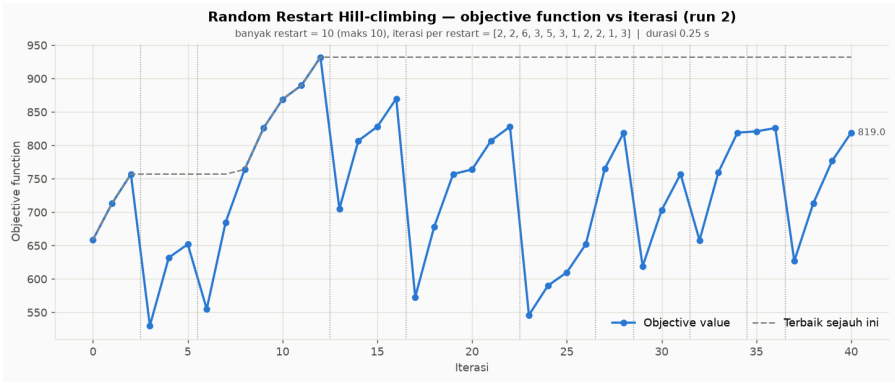
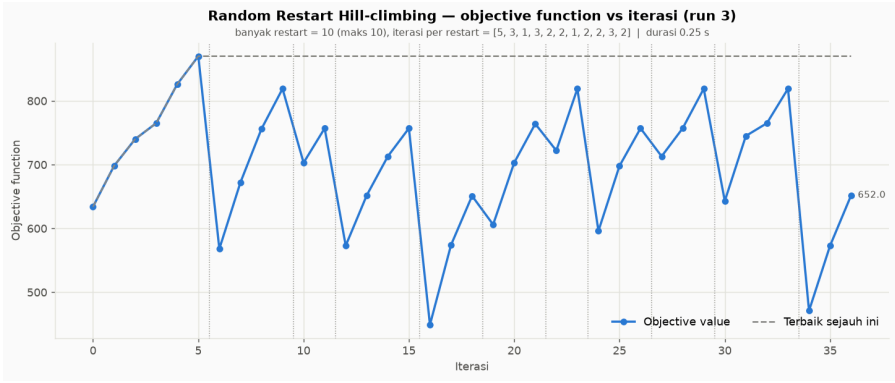
3	

Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.20	0.39	0.23
Jumlah Iterasi	133	264	158
Initial State	578	537	467
Final State	932	936	932

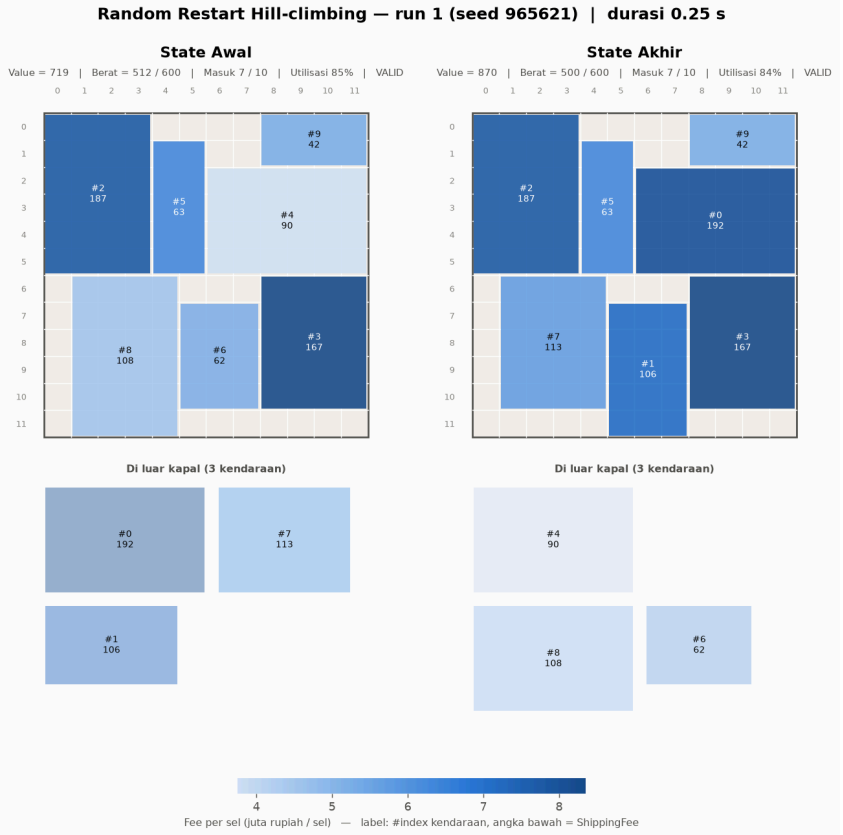
Hill-Climbing with Sideways Move menghasilkan nilai akhir tertinggi di antara semua algoritma, yaitu 932, 936, dan 932, padahal nilai awalnya relatif rendah (578, 537, dan 467). Percobaan 2 bahkan mencapai optimum global 936. Jumlah iterasinya (133, 264, dan 158) jauh lebih banyak dibandingkan Steepest Ascent, karena algoritma tetap boleh berpindah ke neighbor dengan nilai sama ketika tidak ada neighbor yang lebih baik.

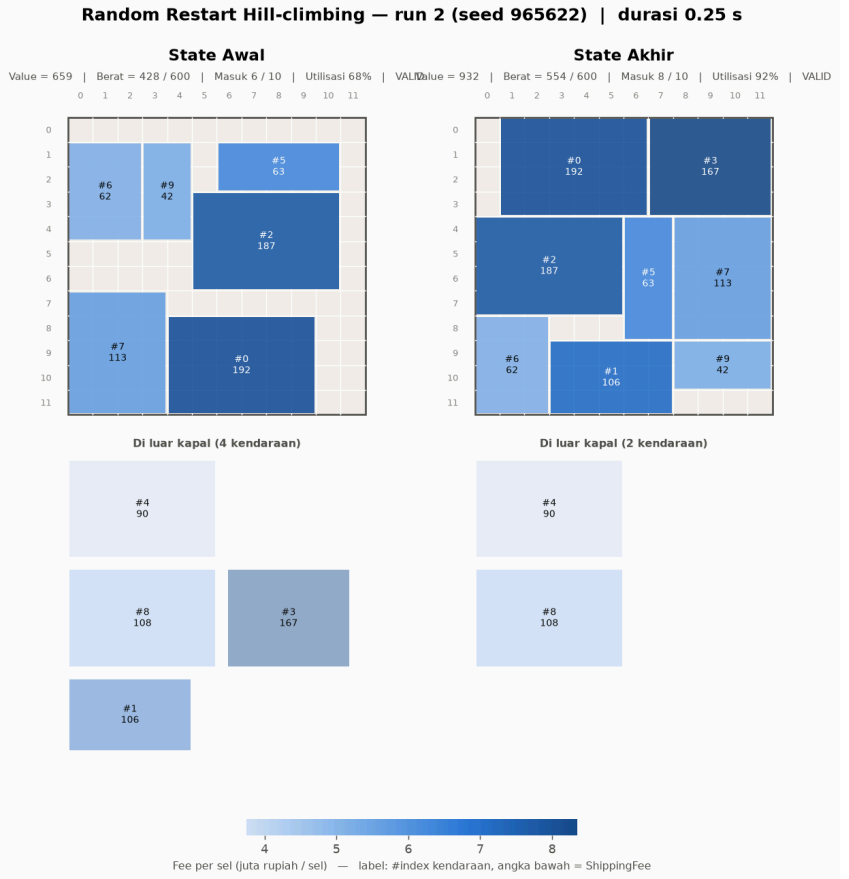
Kemampuan ini penting untuk persoalan ini. Objective function hanya bergantung pada kendaraan yang dimuat, sehingga banyak state bertetangga yang nilainya sama (shoulder). Dengan sideways move, algoritma dapat menggeser atau memutar kendaraan di dataran tersebut sampai terbuka ruang untuk memasukkan kendaraan bernilai lebih tinggi. Durasi (0,20–0,39 s) sebanding dengan jumlah iterasi; percobaan 2 yang iterasinya paling banyak juga paling lama dan menghasilkan nilai terbaik.

C. Random Restart Hill-Climbing

Percobaan	Grafik Nilai Objective Function terhadap Banyak Iterasi
1	
2	
3	

Percobaan	Perbandingan State

1	 Random Restart Hill-climbing — run 1 (seed 965621) | durasi 0.25 s State Awal Value = 719 | Berat = 512 / 600 | Masuk 7 / 10 | Utilisasi 85% | VALID #2 187 #5 63 #9 42 #4 90 #8 108 #6 62 #3 167 State Akhir Value = 870 | Berat = 500 / 600 | Masuk 7 / 10 | Utilisasi 84% | VALID #2 187 #5 63 #9 42 #0 192 #7 113 #1 106 #3 167 Di luar kapal (3 kendaraan) #0 192 #7 113 #1 106 Di luar kapal (3 kendaraan) #4 90 #8 108 #6 62 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

2	 Random Restart Hill-climbing — run 2 (seed 965622) | durasi 0.25 s State Awal Value = 659 | Berat = 428 / 600 | Masuk 6 / 10 | Utilisasi 68% | VALID State Akhir Value = 932 | Berat = 554 / 600 | Masuk 8 / 10 | Utilisasi 92% | VALID #6 62 #9 42 #5 63 #2 187 #7 113 #0 192 #0 192 #3 167 #2 187 #5 63 #7 113 #6 62 #1 106 #9 42 Di luar kapal (4 kendaraan) #4 90 #8 108 #3 167 #1 106 Di luar kapal (2 kendaraan) #4 90 #8 108 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

3	

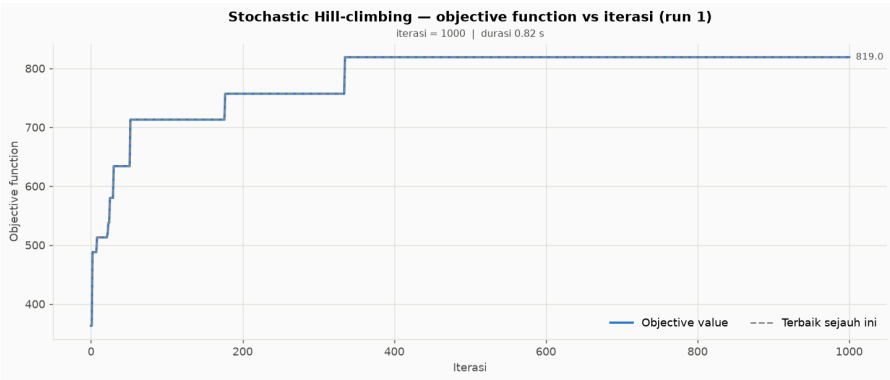
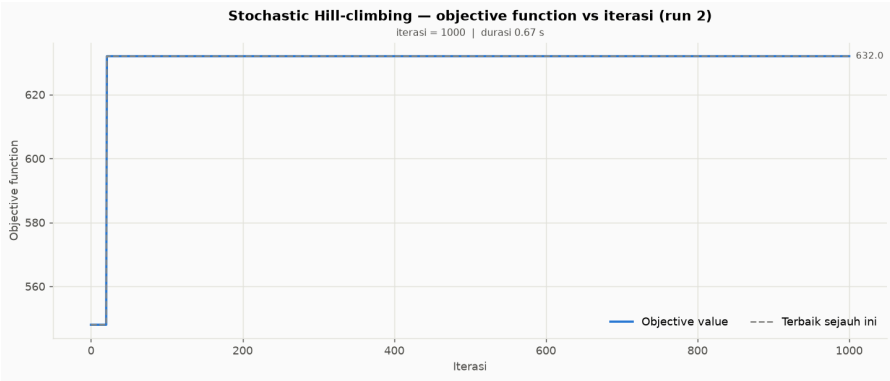
Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.25	0.25	0.25
Jumlah Iterasi	10	10	10
Iterasi per Restart	[3, 2, 1, 5, 4, 2, 3, 2, 1, 6, 0]	[2, 2, 6, 3, 5, 3, 1, 2, 2, 1, 3]	[5, 3, 1, 3, 2, 2, 1, 2, 2, 3, 2]
Initial State	719	659	634
Final State	870	932	870

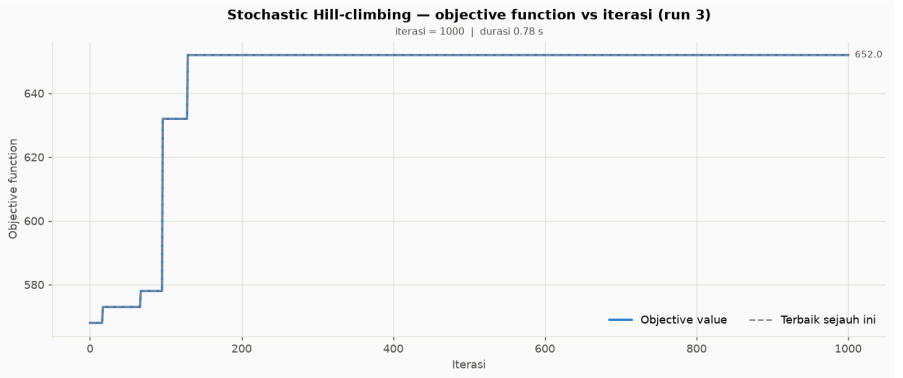
Random Restart Hill-Climbing menjalankan Steepest Ascent sebanyak 10 kali restart dari state awal acak yang berbeda, lalu mengambil hasil terbaik. Setiap restart hanya membutuhkan 0–6 iterasi sebelum stuck, sesuai perilaku Steepest Ascent yang cepat berhenti di optimum lokal.

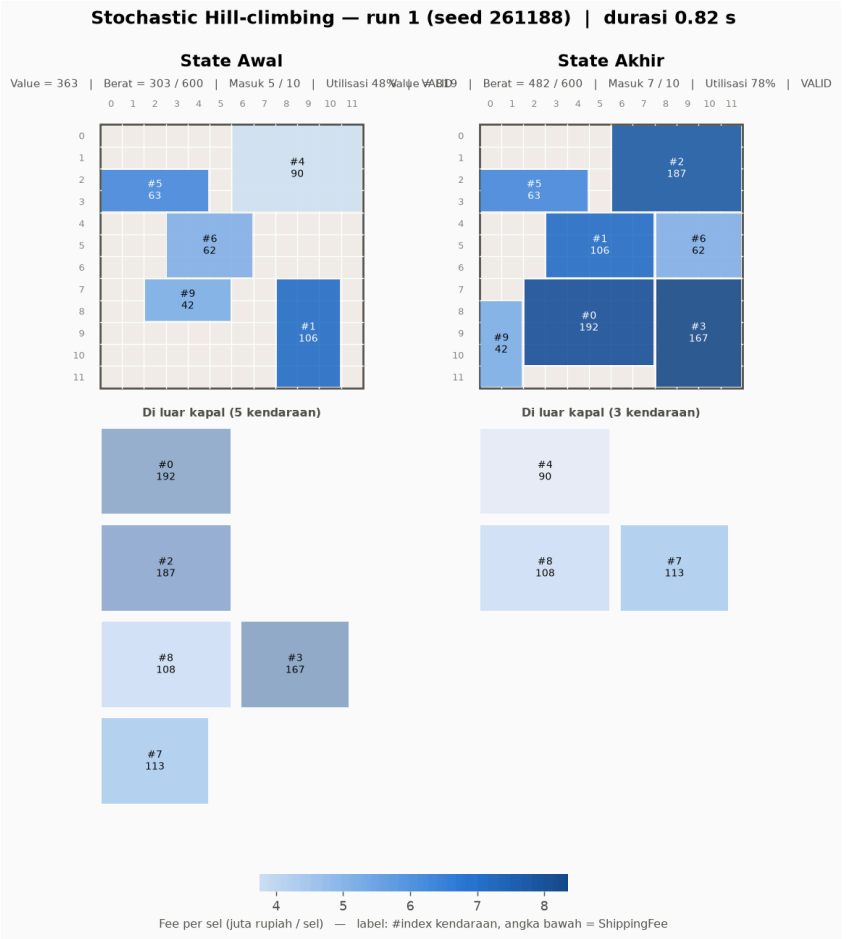
Nilai akhir 870, 932, dan 870 lebih baik daripada Steepest Ascent biasa (632–819) dan meningkat 151, 273, dan 236 dari state awal percobaan.

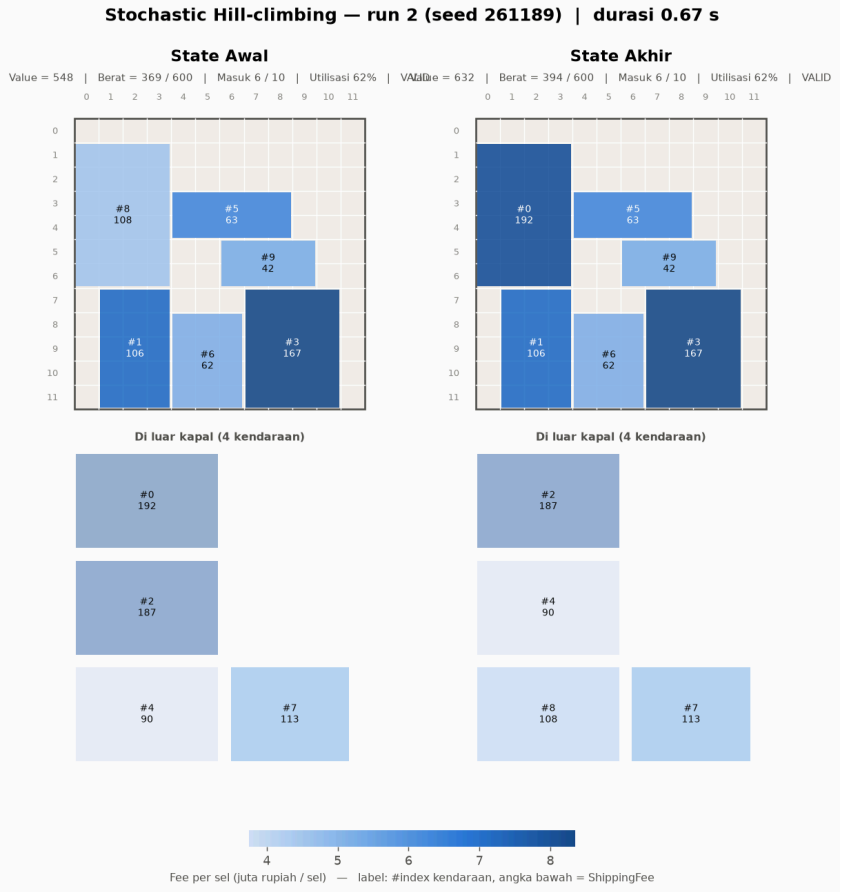
Peningkatan ini terjadi karena semakin banyak state awal yang dicoba, semakin besar peluang salah satunya berada di "lereng" optimum lokal yang tinggi. Percobaan 2 hampir mencapai optimum global (932). Durasi tiap percobaan sekitar 0,25 s, kira-kira sepuluh kali Steepest Ascent, sesuai dengan jumlah restart yang dilakukan.

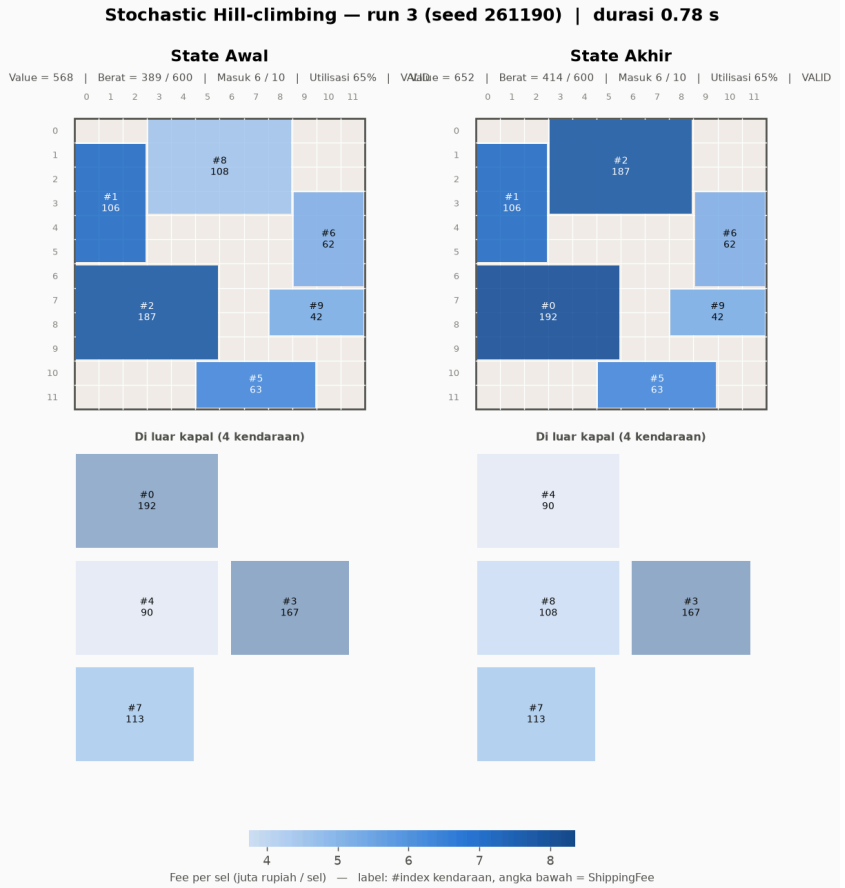
D. Stochastic Hill-Climbing

Percobaan	Grafik Nilai Objective Function terhadap Banyak Iterasi
1	
2	

3	

Percobaan	Perbandingan State
1	

2	 Stochastic Hill-climbing — run 2 (seed 261189) | durasi 0.67 s State Awal Value = 548 | Berat = 369 / 600 | Masuk 6 / 10 | Utilisasi 62% | VALID #8 108 #5 63 #9 42 #1 106 #6 62 #3 167 Di luar kapal (4 kendaraan) #0 192 #2 187 #4 90 #7 113 State Akhir Value = 632 | Berat = 394 / 600 | Masuk 6 / 10 | Utilisasi 62% | VALID #0 192 #5 63 #9 42 #1 106 #6 62 #3 167 Di luar kapal (4 kendaraan) #2 187 #4 90 #8 108 #7 113 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

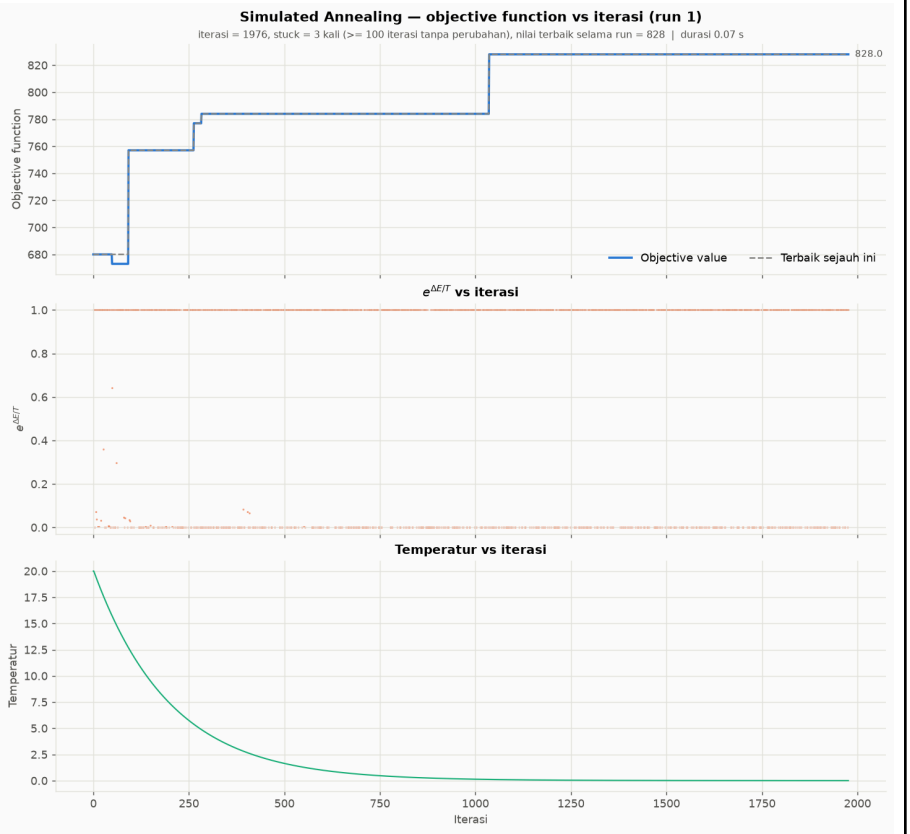
3	

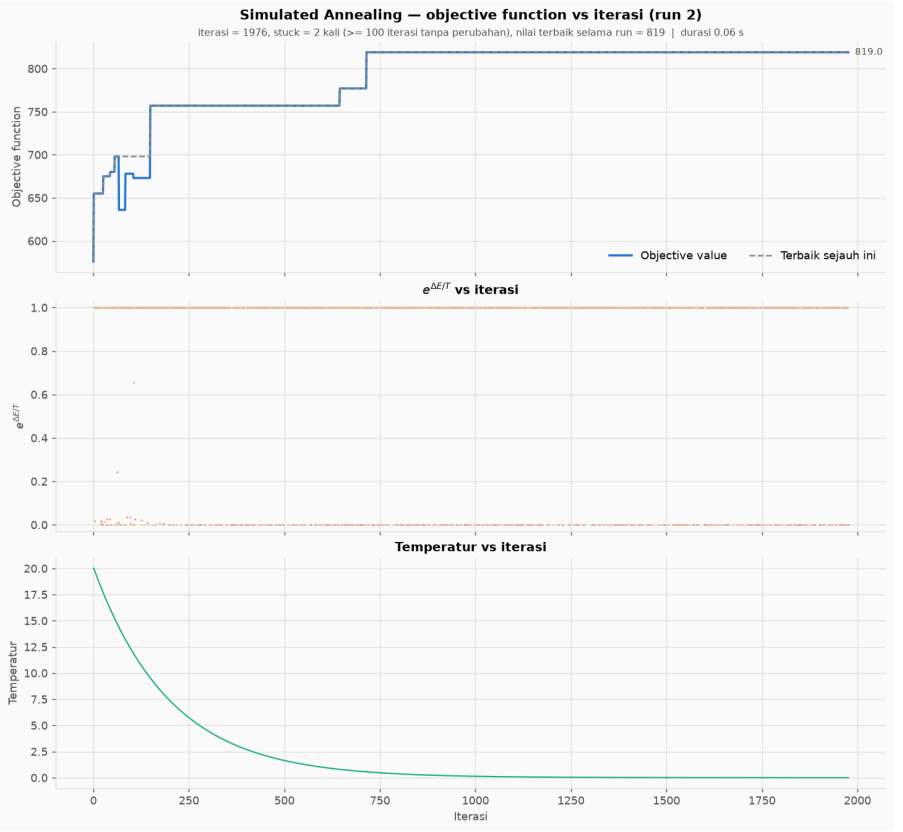
Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.82	0.67	0.78
Jumlah Iterasi	1000	1000	1000
Initial State	363	548	568
Final State	819	632	652

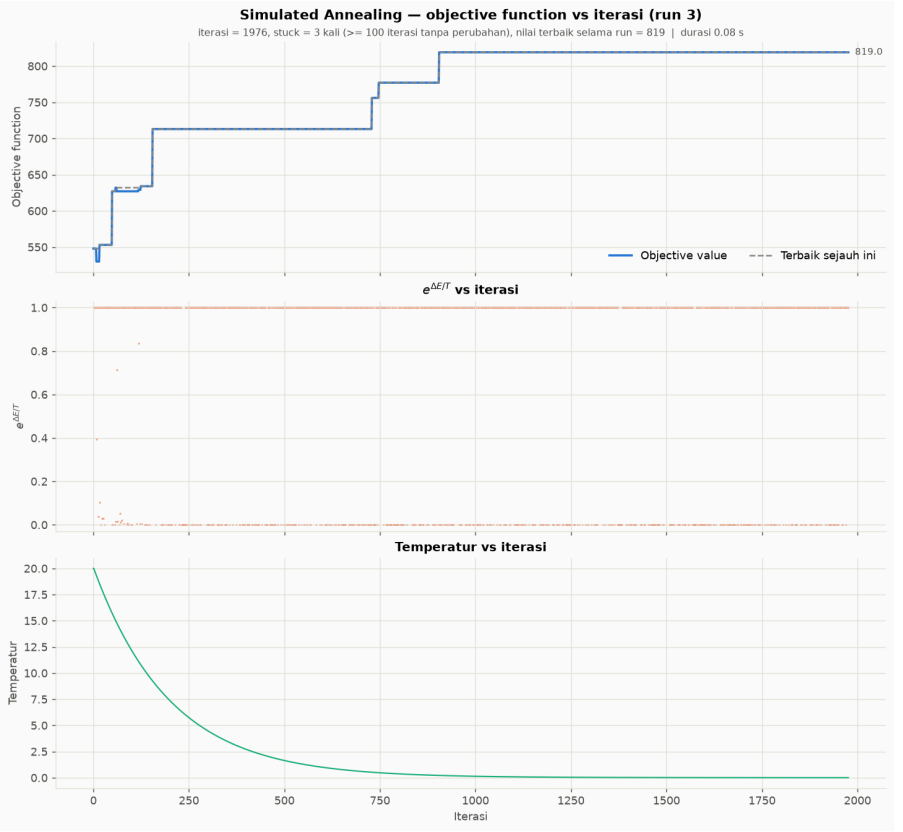
Stochastic Hill-Climbing dijalankan selama 1000 iterasi pada setiap percobaan. Nilai akhir yang diperoleh 819, 632, dan 652 dari nilai awal 363, 548, dan 568. Percobaan 1 mengalami kenaikan terbesar (456), sedangkan percobaan 2 dan 3 hanya naik 84. Pada grafik terlihat bahwa seluruh kenaikan terjadi di awal pencarian (sekitar beberapa ratus iterasi pertama), lalu nilai objective function datar sampai iterasi ke-1000.

Hal ini terjadi karena algoritma hanya memilih satu successor secara acak per iterasi dan hanya menerimanya jika nilainya lebih baik. Setelah mencapai optimum lokal, hampir semua successor acak tidak lebih baik, sehingga iterasi sisanya terbuang. Hasilnya sangat bervariasi antar percobaan dan bergantung pada state awal. Durasinya (0,67–0,82 s) paling lama di antara varian Hill-Climbing karena jumlah iterasinya tetap besar.

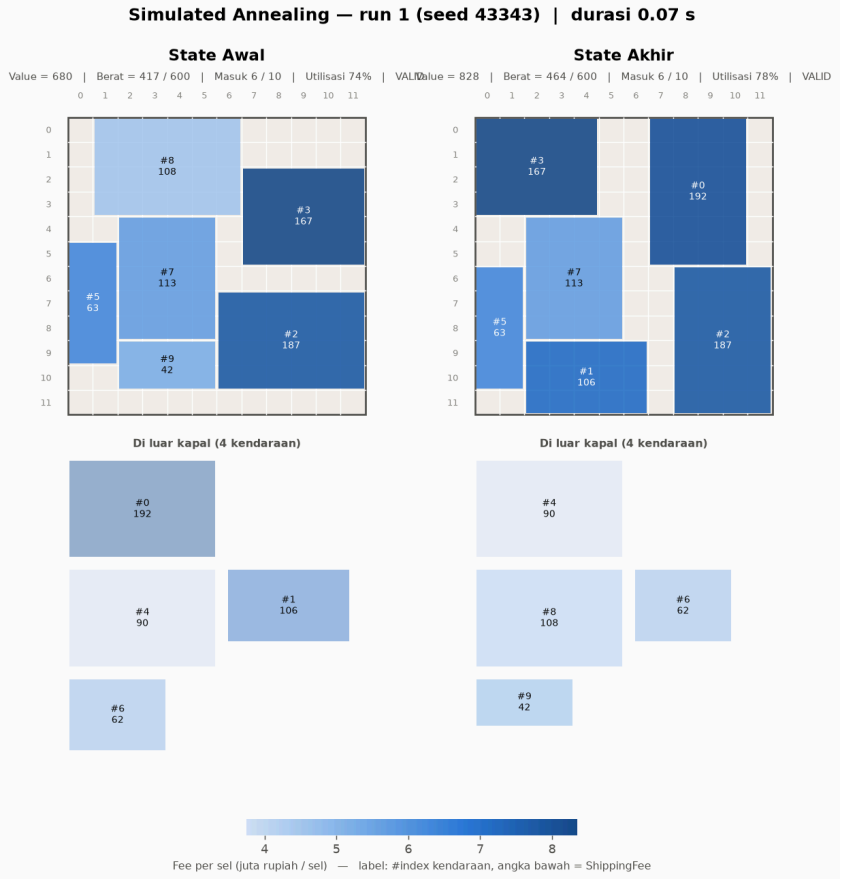
E. Simulated Annealing

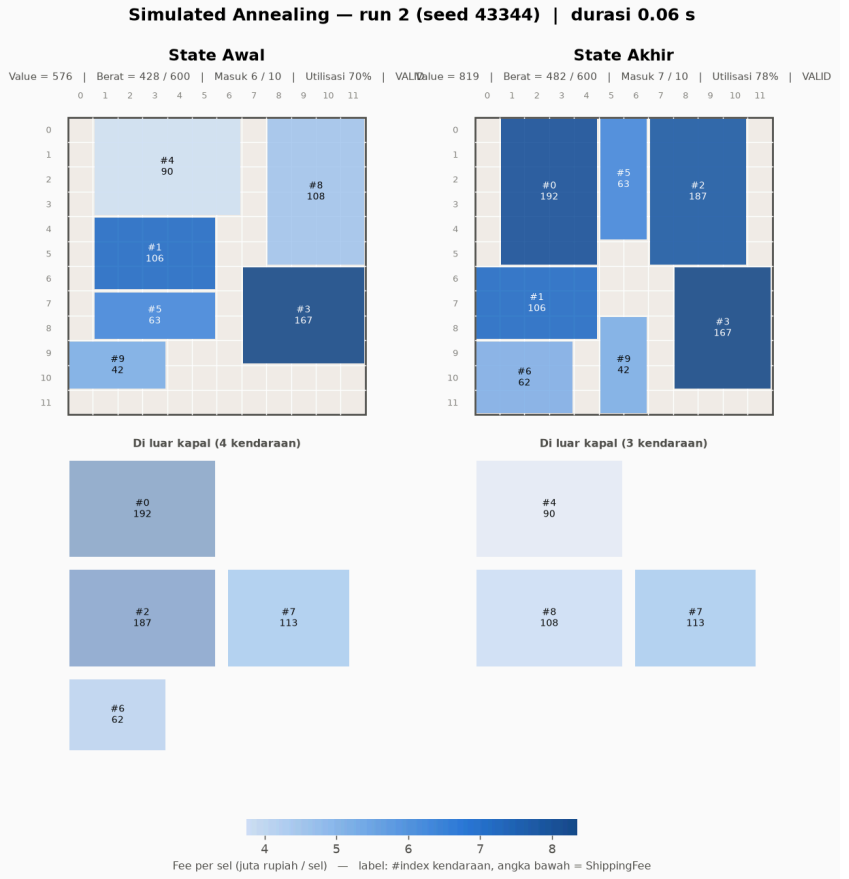
Percobaan	Grafik Nilai Objective Function terhadap Banyak Iterasi
1	

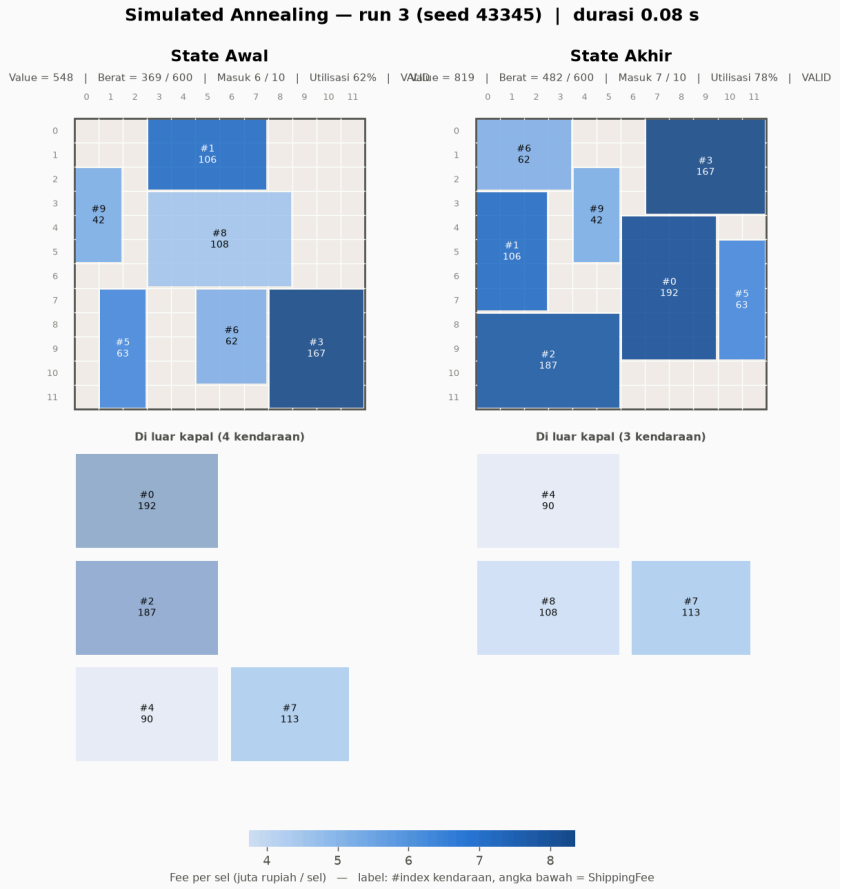
2	 Simulated Annealing — objective function vs iterasi (run 2) Iterasi = 1976, stuck = 2 kali (>= 100 iterasi tanpa perubahan), nilai terbaik selama run = 819 | durasi 0.06 s 819.0 Objective function Objective value Terbaik sejauh ini $e^{\Delta E/T}$ vs iterasi $e^{\Delta E/T}$ Temperatur vs iterasi Temperatur Iterasi

3	

Percobaan	Perbandingan State

1	 Simulated Annealing — run 1 (seed 43343) | durasi 0.07 s State Awal Value = 680 | Berat = 417 / 600 | Masuk 6 / 10 | Utilisasi 74% | VALID #8 108 #3 167 #7 113 #5 63 #2 187 #9 42 State Akhir Value = 828 | Berat = 464 / 600 | Masuk 6 / 10 | Utilisasi 78% | VALID #3 167 #0 192 #7 113 #5 63 #2 187 #1 106 Di luar kapal (4 kendaraan) #0 192 #4 90 #1 106 #6 62 Di luar kapal (4 kendaraan) #4 90 #8 108 #6 62 #9 42 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

2	 Simulated Annealing — run 2 (seed 43344) | durasi 0.06 s State Awal Value = 576 | Berat = 428 / 600 | Masuk 6 / 10 | Utilisasi 70% | VALID #4 90 #8 108 #1 106 #5 63 #3 167 #9 42 State Akhir Value = 819 | Berat = 482 / 600 | Masuk 7 / 10 | Utilisasi 78% | VALID #0 192 #5 63 #2 187 #1 106 #3 167 #6 62 #9 42 Di luar kapal (4 kendaraan) #0 192 #2 187 #7 113 #6 62 Di luar kapal (3 kendaraan) #4 90 #8 108 #7 113 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

3	

Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.07	0.06	0.08
Jumlah Iterasi	1976	1976	1976
Initial State	680	576	548
Final State	828	819	819

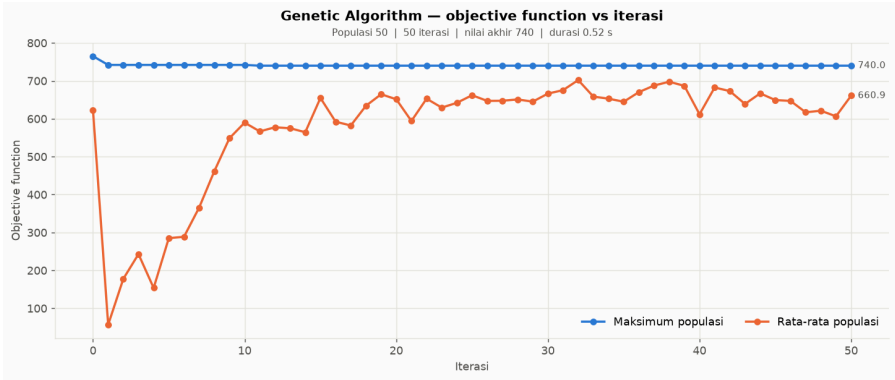
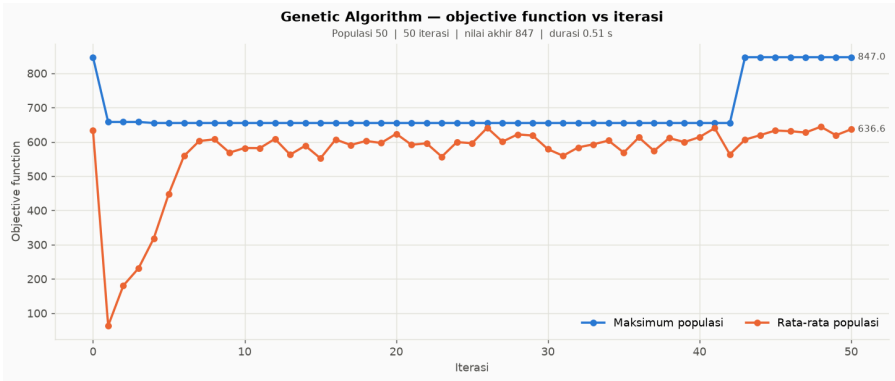
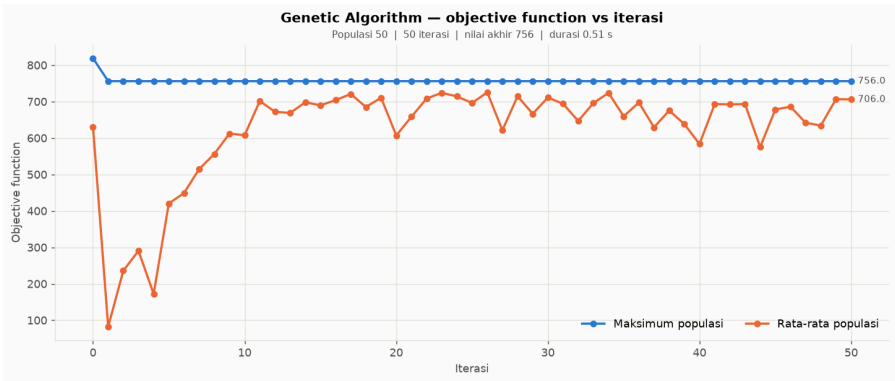
Simulated Annealing selalu berjalan 1976 iterasi, karena jumlah iterasi ditentukan oleh jadwal pendinginan T = 20 · 0,995^(t−1) yang berhenti ketika T < 0,001. Nilai akhir 828, 819, dan 819 diperoleh dari nilai awal 680, 576, dan 548, dengan kenaikan 148, 243, dan 271. Hasilnya lebih konsisten daripada Steepest Ascent dan Stochastic Hill-Climbing, tetapi belum mencapai optimum global.

Grafik temperatur menunjukkan penurunan eksponensial; setelah sekitar 1000 iterasi suhu sudah hampir nol. Pada grafik e^(ΔE/T), sebagian besar titik untuk move yang memperburuk nilai berada dekat 0. Hal ini karena selisih objective function bernilai puluhan sampai ratusan, jauh lebih besar dari T maksimum 20, sehingga move yang memperburuk hampir tidak pernah diterima. Akibatnya Simulated Annealing berperilaku mirip Stochastic Hill-Climbing dan grafik nilai objective function naik bertahap tanpa banyak penurunan. Durasinya sangat singkat (0,06–0,08 s) karena setiap iterasi hanya mengevaluasi satu neighbor.

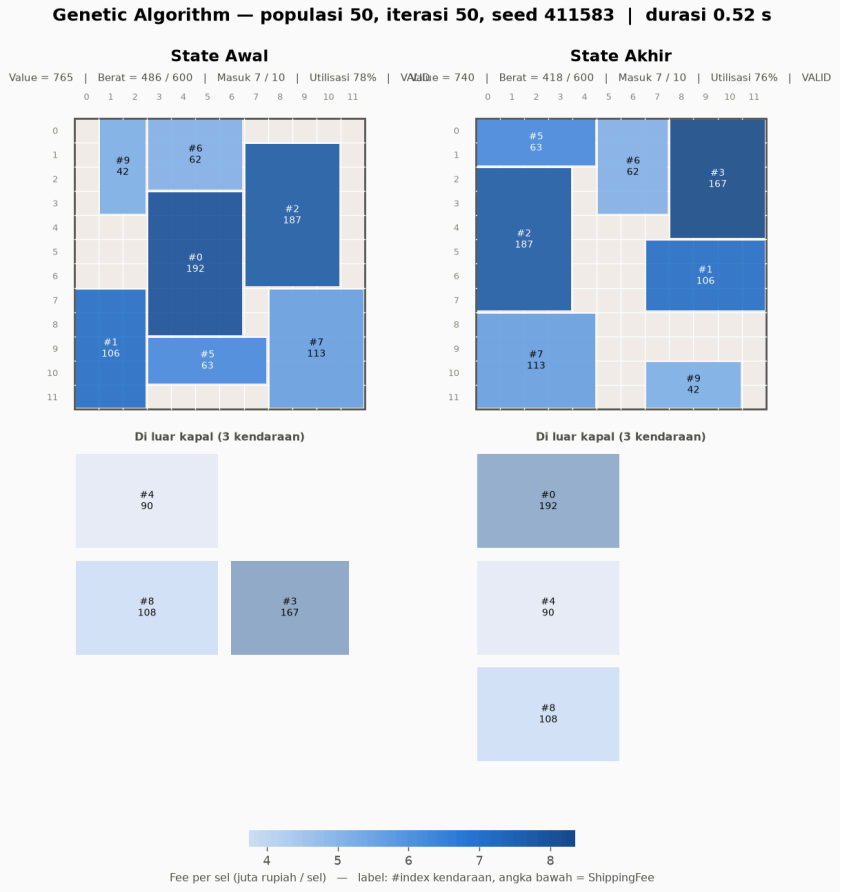
F. Genetic Algorithm

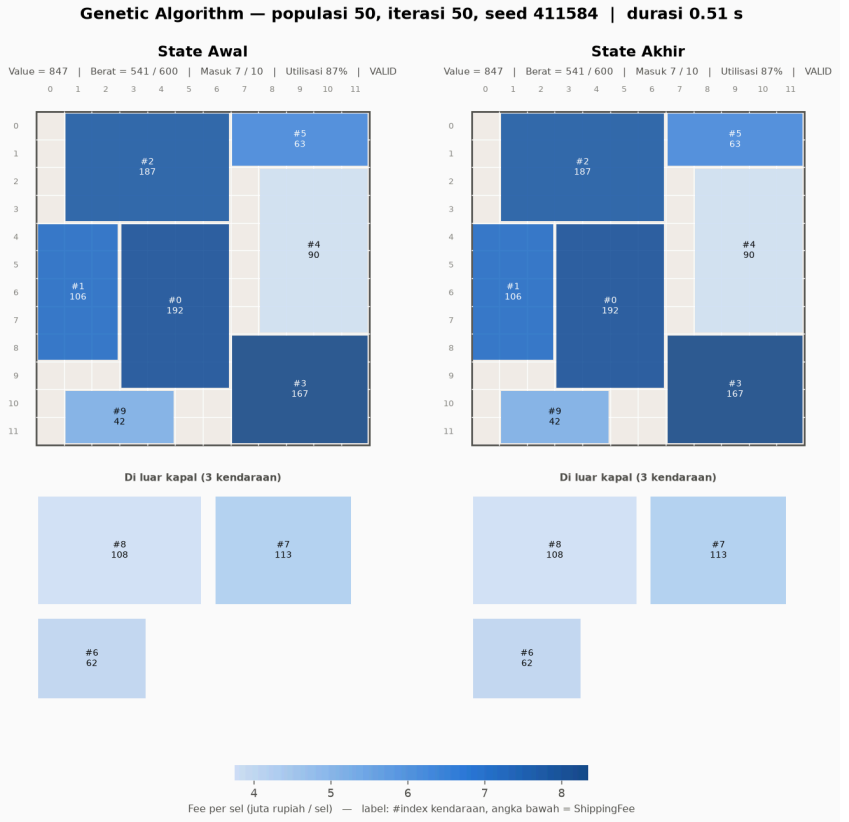
1. Variasi Literasi

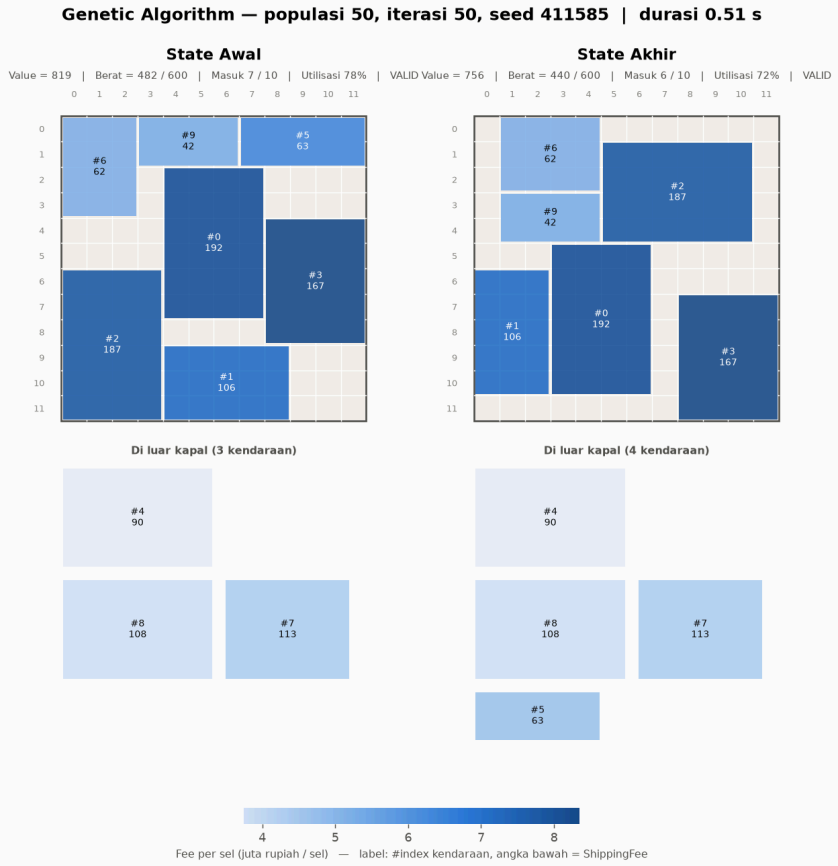
   a) Populasi 50, Iterasi 50

Percobaan	Grafik Nilai Objective Function terhadap Banyak Iterasi
1	
2	
3	

Percobaan	Perbandingan State

1	 Genetic Algorithm — populasi 50, iterasi 50, seed 411583 | durasi 0.52 s State Awal Value = 765 | Berat = 486 / 600 | Masuk 7 / 10 | Utilisasi 78% | VALID #9 42 #6 62 #2 187 #0 192 #1 106 #5 63 #7 113 Di luar kapal (3 kendaraan) #4 90 #8 108 #3 167 State Akhir Value = 740 | Berat = 418 / 600 | Masuk 7 / 10 | Utilisasi 76% | VALID #5 63 #6 62 #3 167 #2 187 #1 106 #7 113 #9 42 Di luar kapal (3 kendaraan) #0 192 #4 90 #8 108 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

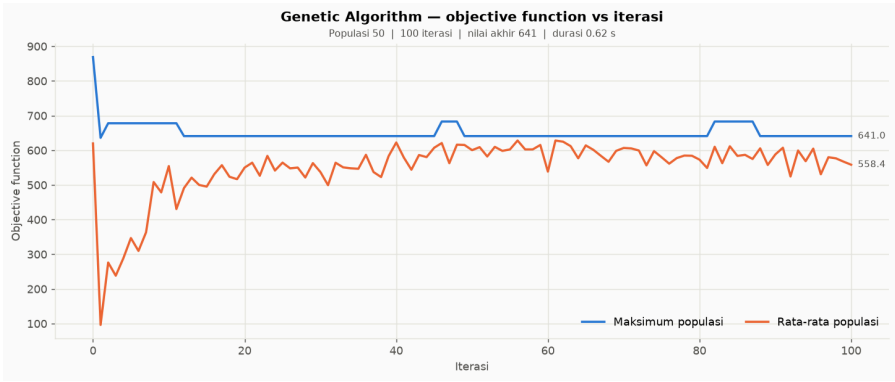
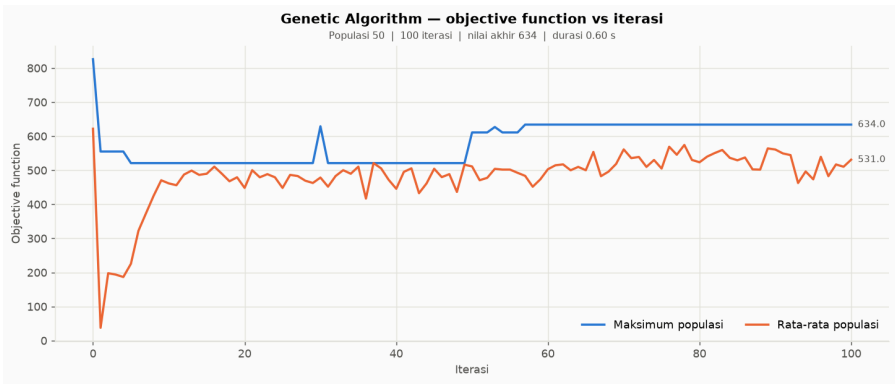
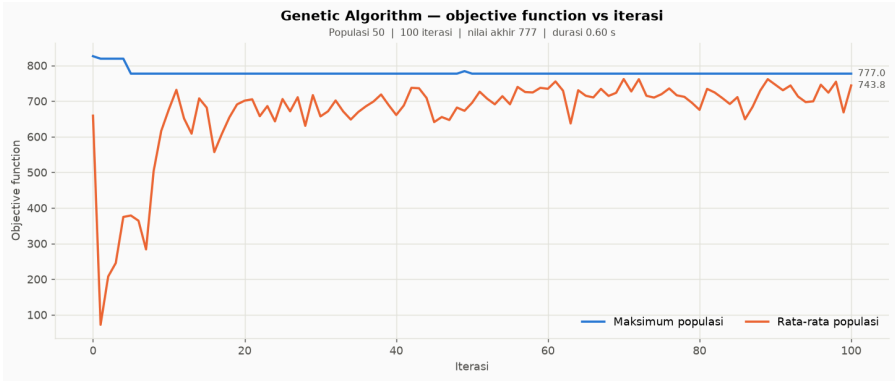
2	 Genetic Algorithm — populasi 50, iterasi 50, seed 411584 | durasi 0.51 s State Awal Value = 847 | Berat = 541 / 600 | Masuk 7 / 10 | Utilisasi 87% | VALID State Akhir Value = 847 | Berat = 541 / 600 | Masuk 7 / 10 | Utilisasi 87% | VALID #2 187 #5 63 #4 90 #1 106 #0 192 #3 167 #9 42 Di luar kapal (3 kendaraan) #8 108 #7 113 #6 62 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

3	

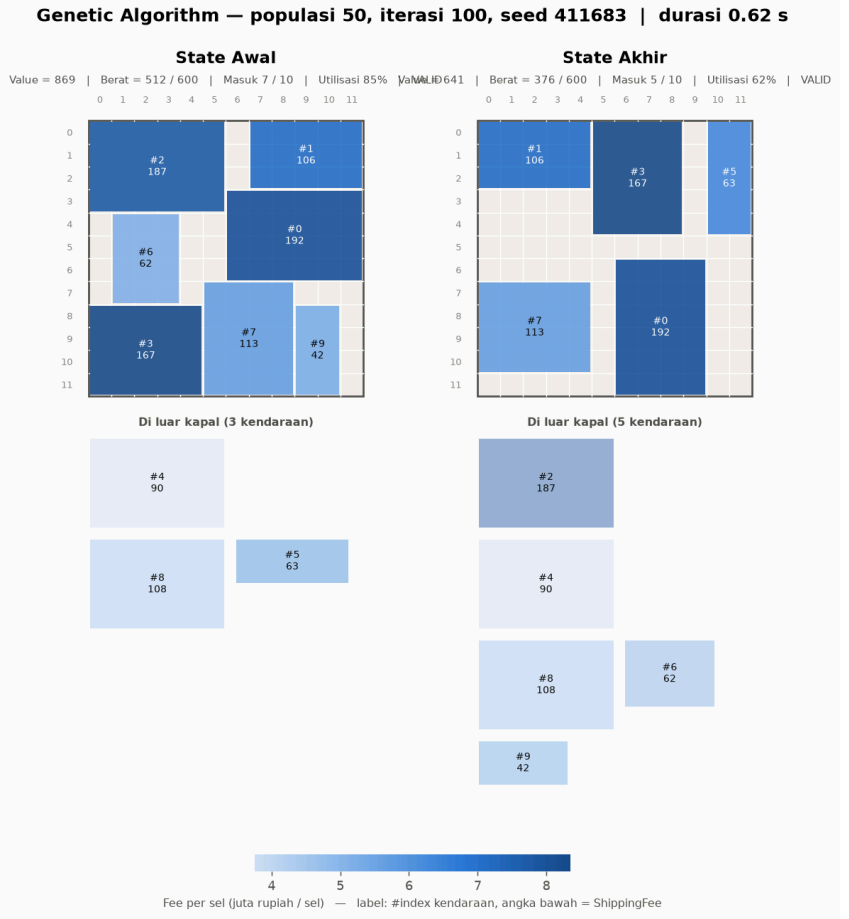
Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.52	0.51	0.51
Jumlah Iterasi	50	50	50
Initial State	765	847	819
Final State	740	847	756

b) Populasi 50, Iterasi 100

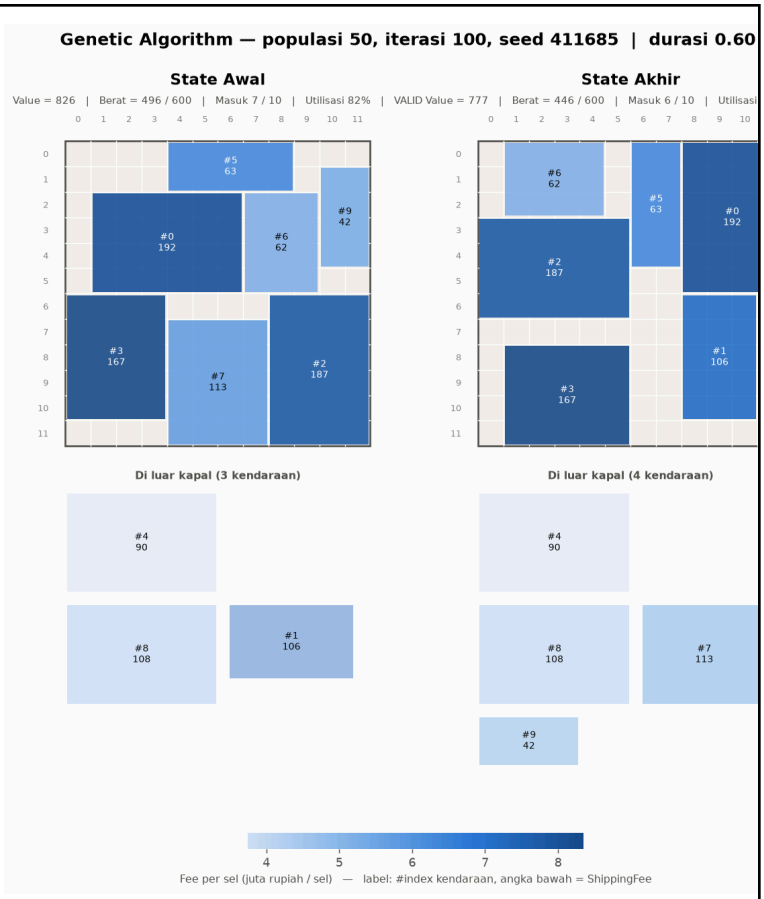
Percobaan	**Grafik Nilai Objective Function terhadap Banyak Iterasi**

1	
2	
3	

Percobaan	Perbandingan State

1	 Genetic Algorithm — populasi 50, iterasi 100, seed 411683 | durasi 0.62 s State Awal Value = 869 | Berat = 512 / 600 | Masuk 7 / 10 | Utilisasi 85% | VALID #2 187 #1 106 #0 192 #6 62 #7 113 #9 42 #3 167 State Akhir Value = 641 | Berat = 376 / 600 | Masuk 5 / 10 | Utilisasi 62% | VALID #1 106 #3 167 #5 63 #7 113 #0 192 Di luar kapal (3 kendaraan) #4 90 #8 108 #5 63 Di luar kapal (5 kendaraan) #2 187 #4 90 #8 108 #6 62 #9 42 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

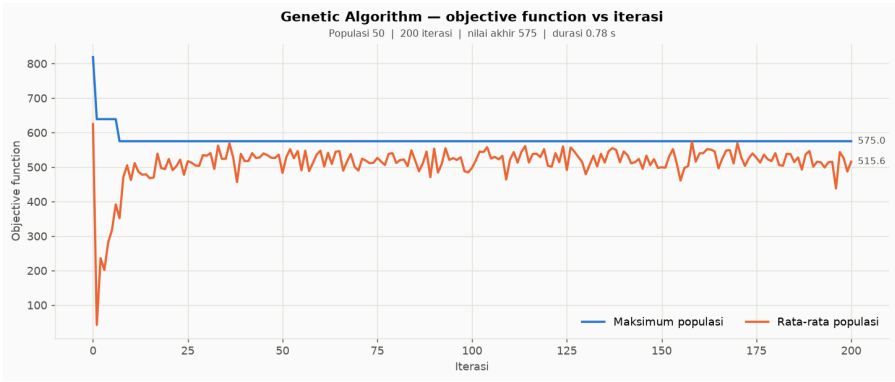
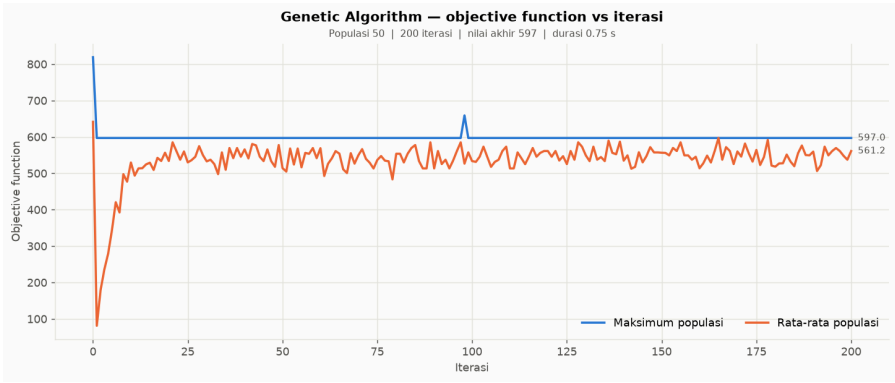
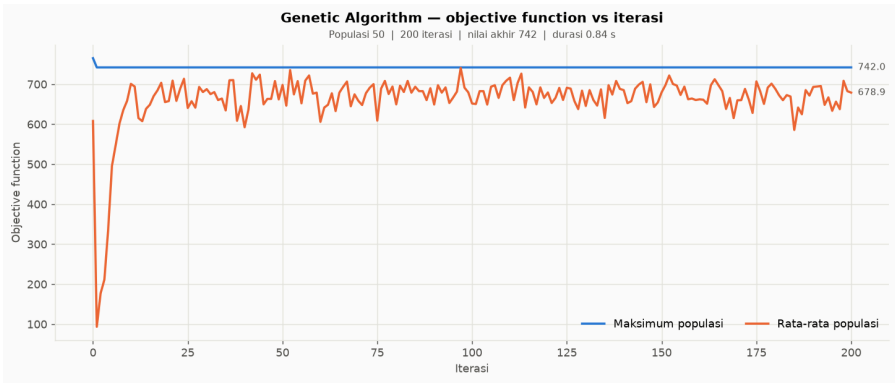
2	 Genetic Algorithm — populasi 50, iterasi 100, seed 411684 | durasi 0.60 s State Awal Value = 826 | Berat = 496 / 600 | Masuk 7 / 10 | Utilisasi 82% | VALID #5 63 #7 113 #9 42 #0 192 #3 167 #2 187 #6 62 State Akhir Value = 634 | Berat = 360 / 600 | Masuk 6 / 10 | Utilisasi 65% | VALID #3 167 #7 113 #2 187 #6 62 #9 42 #5 63 Di luar kapal (3 kendaraan) #4 90 #8 108 #1 106 Di luar kapal (4 kendaraan) #0 192 #4 90 #8 108 #1 106 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

3	

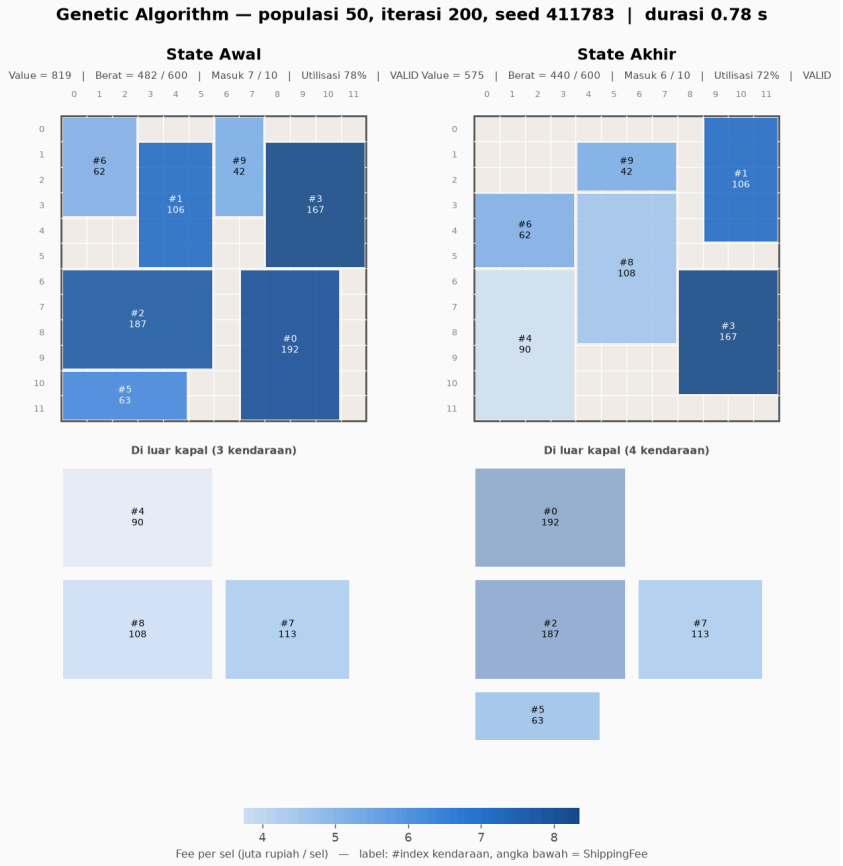
Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.62	0.60	0.60
Jumlah Iterasi	100	100	100
Initial State	869	826	826
Final State	641	634	777

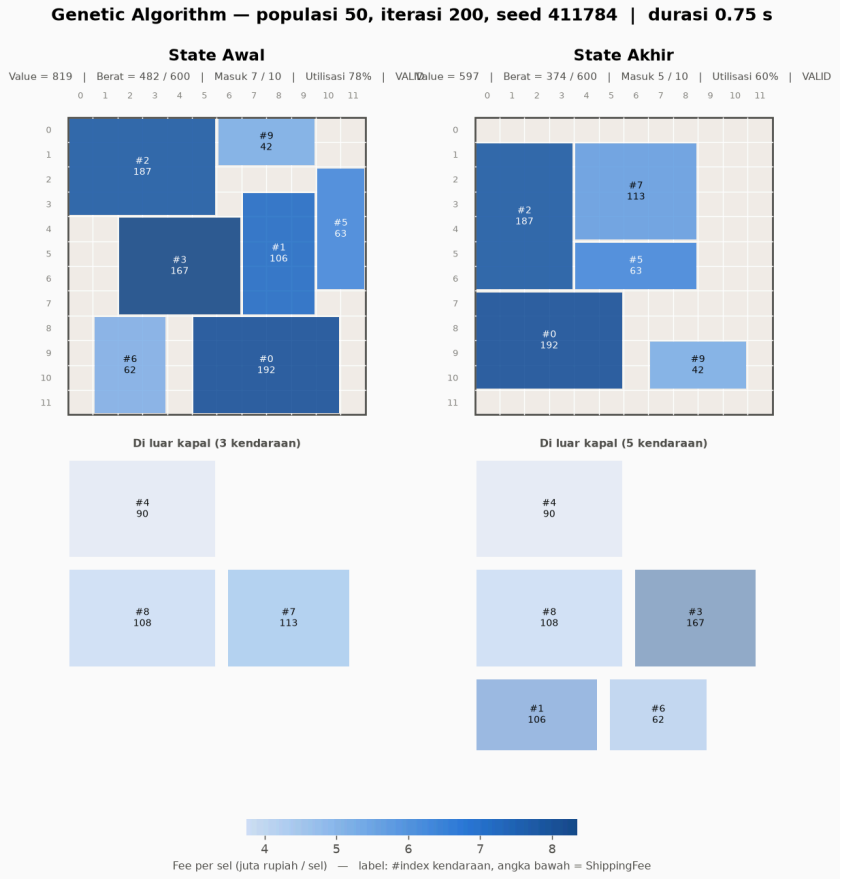
c) Populasi 50, Iterasi 200

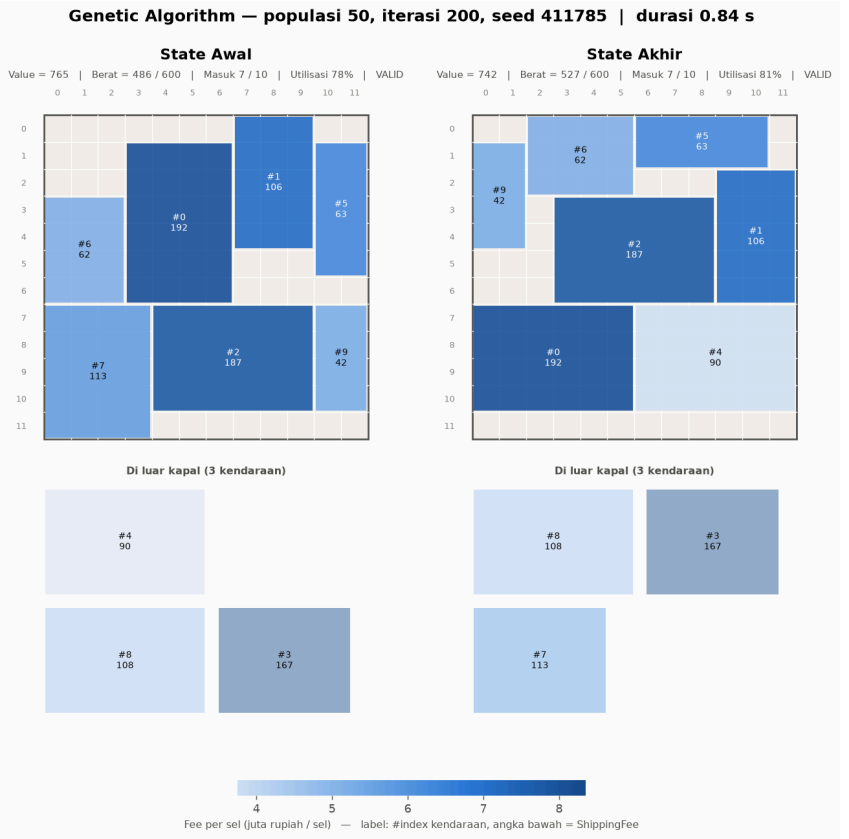
Percobaan	**Grafik Nilai Objective Function terhadap Banyak Iterasi**

1	
2	
3	

Percobaan	Perbandingan State

1	 Genetic Algorithm — populasi 50, iterasi 200, seed 411783 | durasi 0.78 s State Awal Value = 819 | Berat = 482 / 600 | Masuk 7 / 10 | Utilisasi 78% | VALID State Akhir Value = 575 | Berat = 440 / 600 | Masuk 6 / 10 | Utilisasi 72% | VALID #6 62 #1 106 #9 42 #3 167 #2 187 #0 192 #5 63 #9 42 #1 106 #6 62 #8 108 #4 90 #3 167 Di luar kapal (3 kendaraan) Di luar kapal (4 kendaraan) #4 90 #8 108 #7 113 #0 192 #2 187 #7 113 #5 63 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

2	 Genetic Algorithm — populasi 50, iterasi 200, seed 411784 | durasi 0.75 s State Awal Value = 819 | Berat = 482 / 600 | Masuk 7 / 10 | Utilisasi 78% | VALID State Akhir Value = 597 | Berat = 374 / 600 | Masuk 5 / 10 | Utilisasi 60% | VALID #2 187 #9 42 #5 63 #3 167 #1 106 #6 62 #0 192 #7 113 #0 192 Di luar kapal (3 kendaraan) Di luar kapal (5 kendaraan) #4 90 #8 108 #7 113 #3 167 #1 106 #6 62 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

3	

Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.78	0.75	0.84
Jumlah Iterasi	200	200	200
Initial State	819	819	765
Final State	575	597	742

Analisis Variasi Iterasi

Pada variasi ini jumlah populasi ditetapkan 50, sedangkan banyak iterasi divariasikan menjadi 50, 100, dan 200. Ringkasan hasilnya ditunjukkan pada tabel berikut.

Konfigurasi	Nilai awal terbaik	Nilai akhir	Rata-rata akhir	Rata-rata durasi (s)

Iterasi 50	765, 847, 819	740, 847, 756	781,0	0,51
Iterasi 100	869, 826, 826	641, 634, 777	684,0	0,61
Iterasi 200	819, 819, 765	575, 597, 742	638,0	0,79

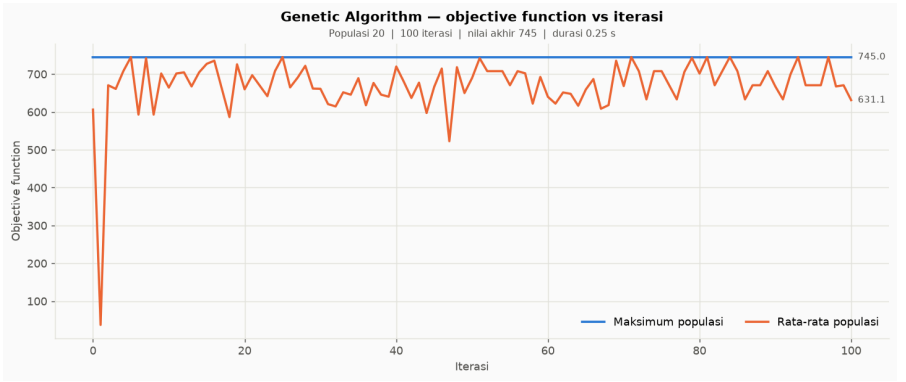
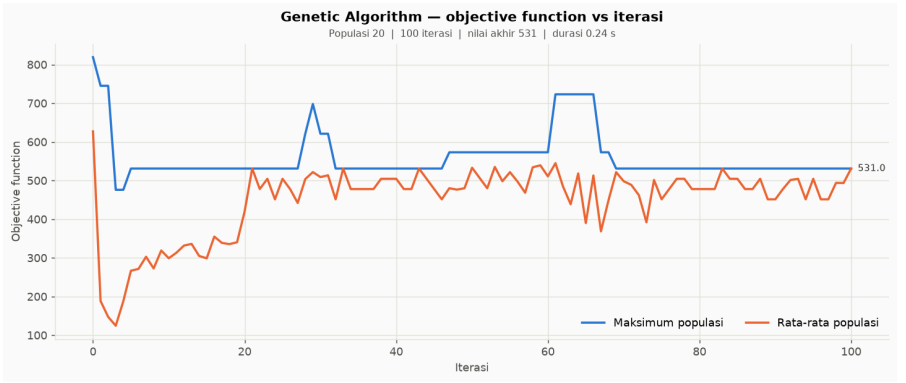
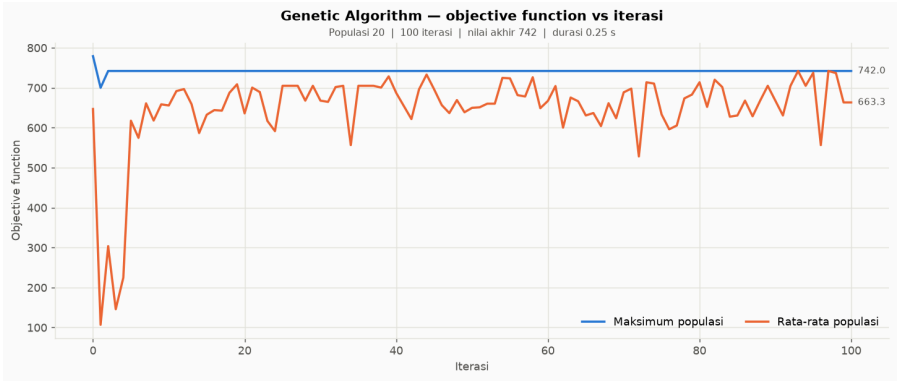
Tabel ringkasan variasi iterasi Genetic Algorithm (50 populasi)

Hasil menunjukkan tren yang berlawanan dengan dugaan awal: semakin banyak iterasi, rata-rata nilai akhir justru semakin rendah, dari 781,0 (50 iterasi) menjadi 684,0 (100 iterasi) dan 638,0 (200 iterasi). Pada hampir semua percobaan, nilai akhir lebih rendah daripada individu terbaik populasi awal. Penurunan terbesar terjadi pada 200 iterasi (244 pada percobaan 1), sedangkan satu-satunya percobaan yang tidak turun adalah percobaan 2 pada 50 iterasi (tetap 847). Grafik percobaan tersebut menunjukkan individu terbaik sempat hilang lalu ditemukan kembali menjelang akhir pencarian.

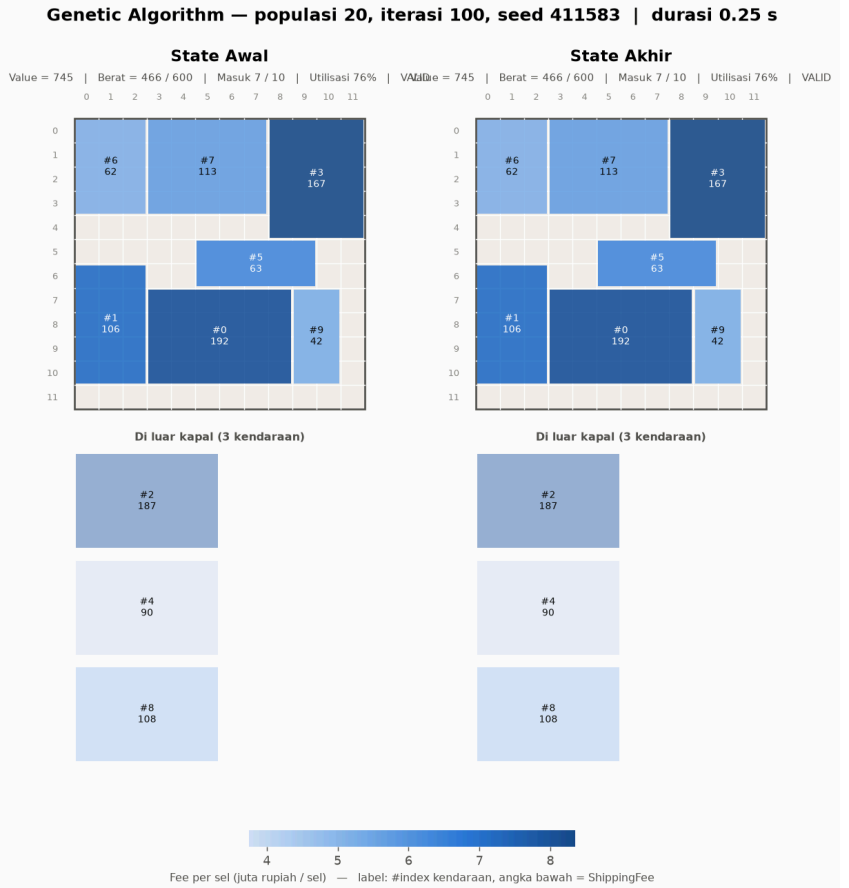
Grafik rata-rata populasi pada setiap konfigurasi turun tajam pada generasi pertama, karena banyak anak hasil one-point crossover yang bertumpuk atau melebihi kapasitas sehingga bernilai 0. Setelah itu rata-rata naik kembali karena roulette selection lebih sering memilih individu valid, tetapi nilai maksimum populasi cenderung datar atau turun. Karena seluruh populasi diganti oleh anak setiap generasi tanpa elitisme, setiap generasi tambahan merupakan kesempatan baru bagi individu terbaik untuk tidak terpilih atau rusak akibat crossover dan mutasi. Durasi bertambah seiring banyak iterasi (0,51 s sampai 0,79 s), tetapi tidak diikuti peningkatan kualitas solusi.

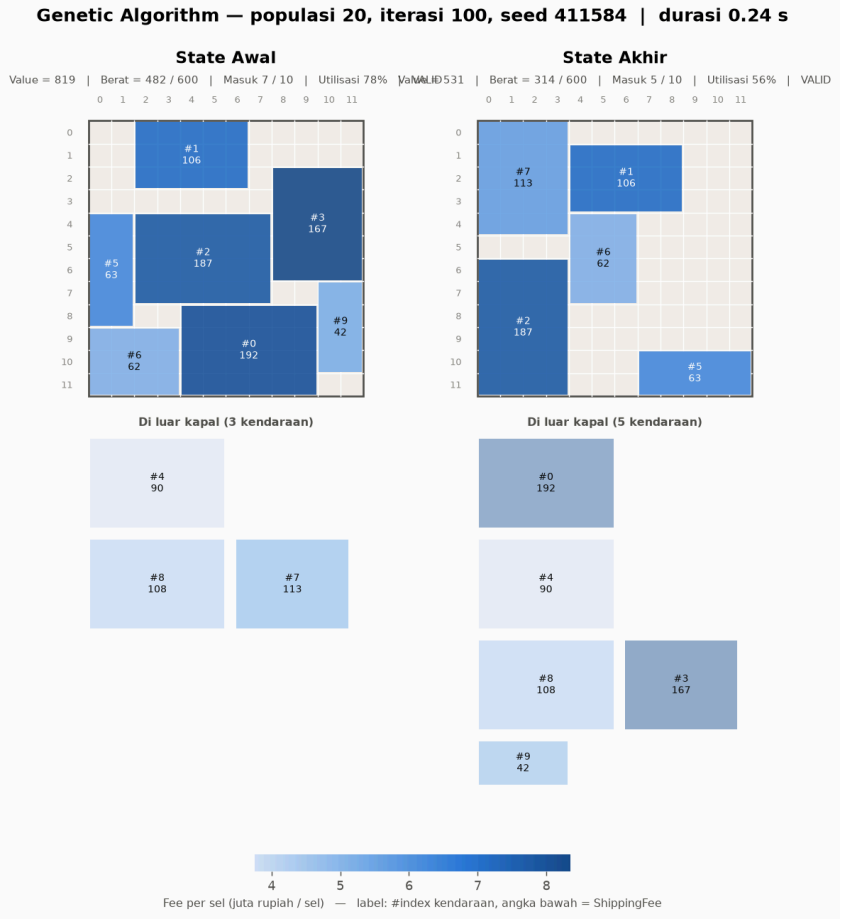
2. Variasi Populasi
    a) Populasi 20, Iterasi 100

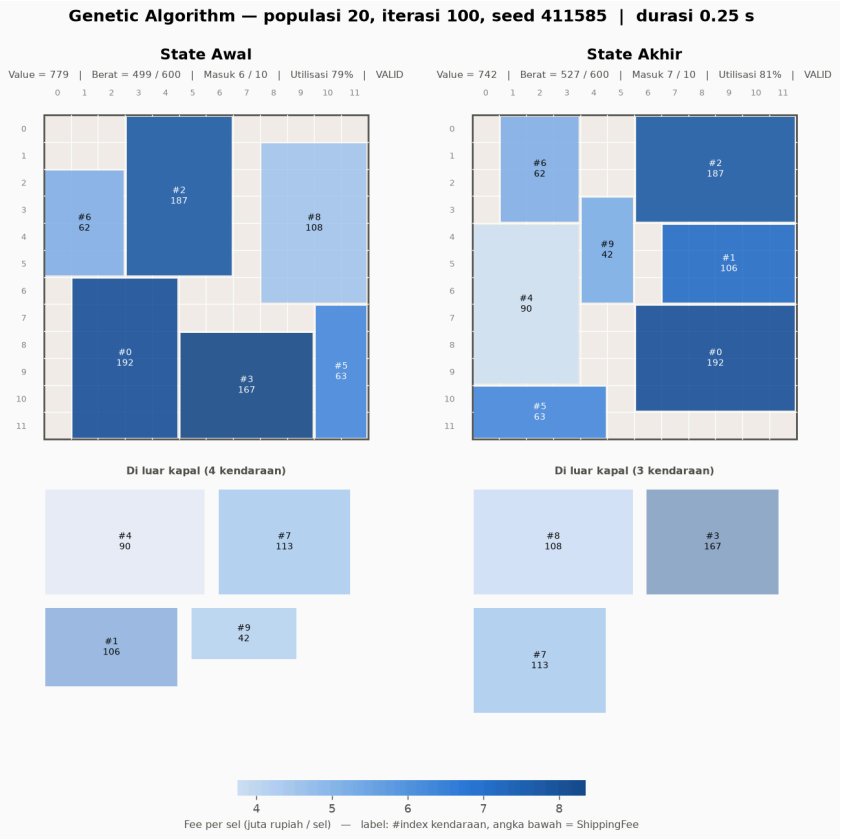
Percobaan	**Grafik Nilai Objective Function terhadap Banyak Iterasi**

1	
2	
3	

Percobaan	Perbandingan State

1	 Genetic Algorithm — populasi 20, iterasi 100, seed 411583 | durasi 0.25 s State Awal Value = 745 | Berat = 466 / 600 | Masuk 7 / 10 | Utilisasi 76% | VALID State Akhir Value = 745 | Berat = 466 / 600 | Masuk 7 / 10 | Utilisasi 76% | VALID #6 62 #7 113 #3 167 #5 63 #1 106 #0 192 #9 42 Di luar kapal (3 kendaraan) #2 187 #4 90 #8 108 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

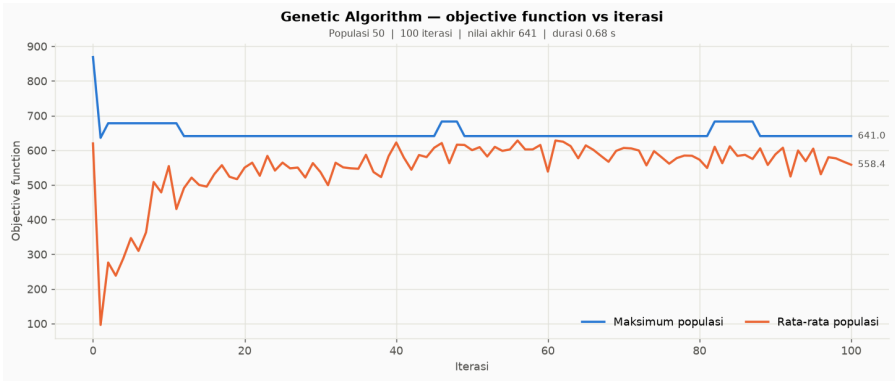
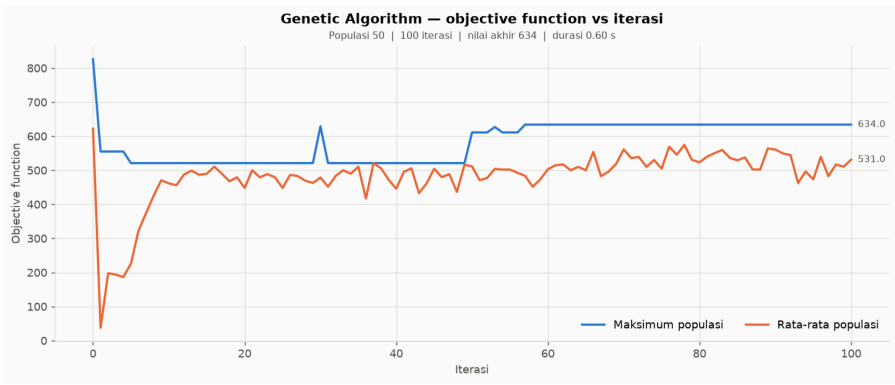
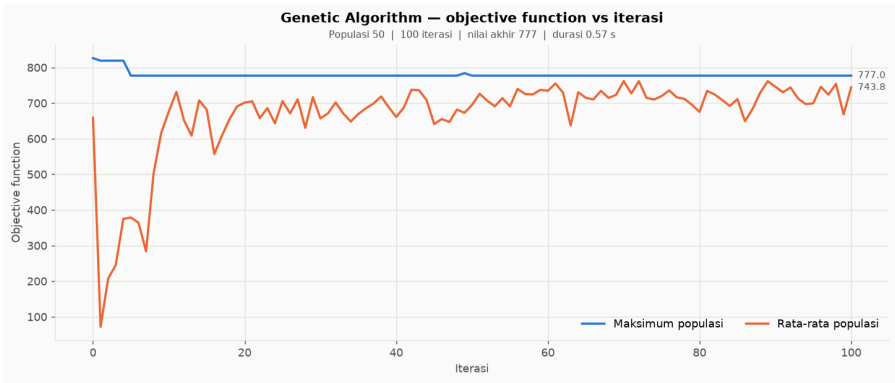
2	 Genetic Algorithm — populasi 20, iterasi 100, seed 411584 | durasi 0.24 s State Awal Value = 819 | Berat = 482 / 600 | Masuk 7 / 10 | Utilisasi 78% | VALID #1 106 #3 167 #5 63 #2 187 #9 42 #6 62 #0 192 State Akhir Value = 531 | Berat = 314 / 600 | Masuk 5 / 10 | Utilisasi 56% | VALID #7 113 #1 106 #6 62 #2 187 #5 63 Di luar kapal (3 kendaraan) #4 90 #8 108 #7 113 Di luar kapal (5 kendaraan) #0 192 #4 90 #8 108 #3 167 #9 42 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

3	

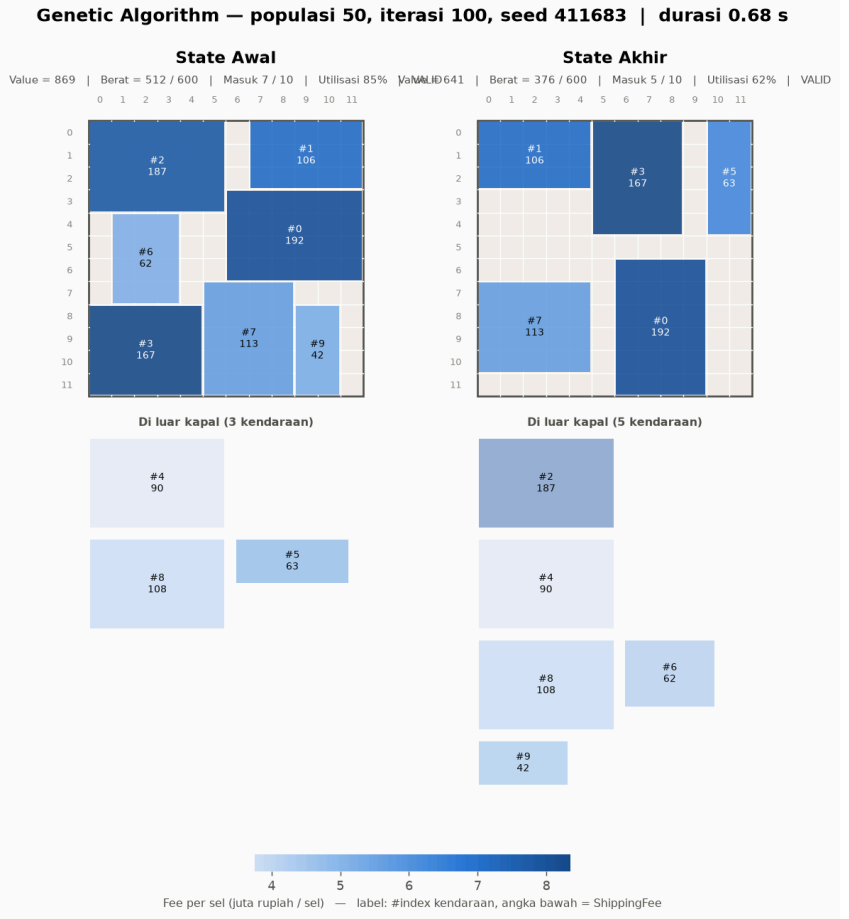
Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.25	0.24	0.25
Jumlah Iterasi	100	100	100
Initial State	745	819	779
Final State	745	531	742

b) Populasi 50, Iterasi 100

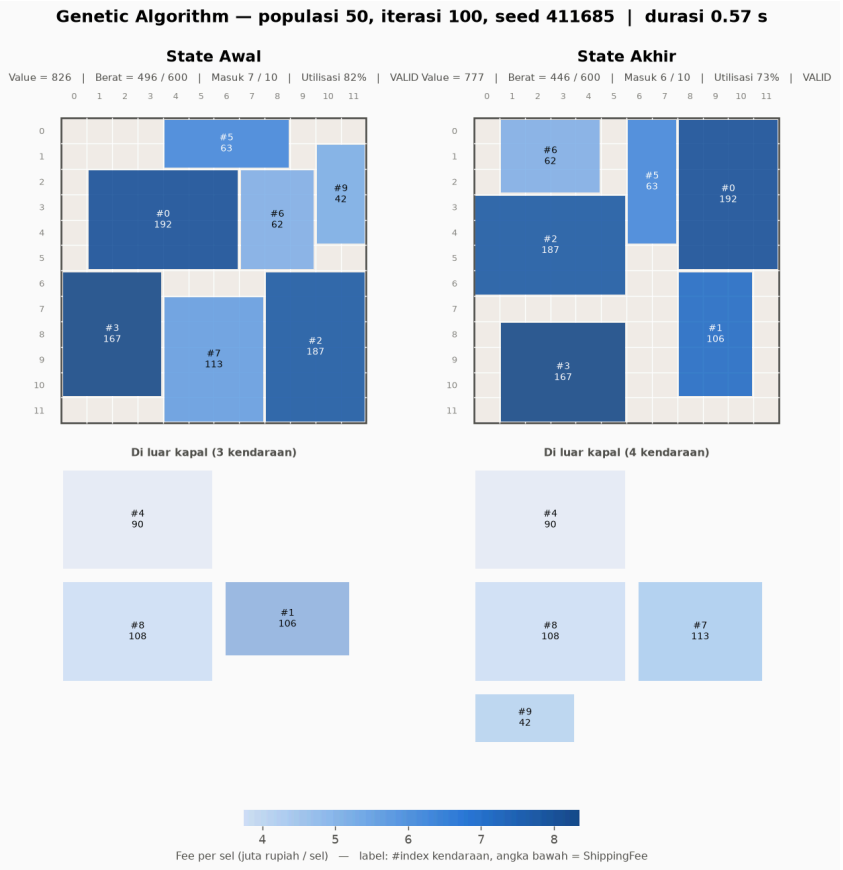
Percobaan	**Grafik Nilai Objective Function terhadap Banyak Iterasi**

1	
2	
3	

Percobaan	Perbandingan State

1	 Genetic Algorithm — populasi 50, iterasi 100, seed 411683 | durasi 0.68 s State Awal Value = 869 | Berat = 512 / 600 | Masuk 7 / 10 | Utilisasi 85% | VALID #2 187 #1 106 #0 192 #6 62 #3 167 #7 113 #9 42 State Akhir Value = 641 | Berat = 376 / 600 | Masuk 5 / 10 | Utilisasi 62% | VALID #1 106 #3 167 #5 63 #7 113 #0 192 Di luar kapal (3 kendaraan) #4 90 #8 108 #5 63 Di luar kapal (5 kendaraan) #2 187 #4 90 #8 108 #6 62 #9 42 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

2	 Genetic Algorithm — populasi 50, iterasi 100, seed 411684 | durasi 0.60 s State Awal Value = 826 | Berat = 496 / 600 | Masuk 7 / 10 | Utilisasi 82% | VALID State Akhir Value = 634 | Berat = 360 / 600 | Masuk 6 / 10 | Utilisasi 65% | VALID #5 63 #7 113 #9 42 #0 192 #3 167 #2 187 #6 62 #3 167 #7 113 #2 187 #6 62 #9 42 #5 63 Di luar kapal (3 kendaraan) Di luar kapal (4 kendaraan) #4 90 #8 108 #1 106 #0 192 #4 90 #8 108 #1 106 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

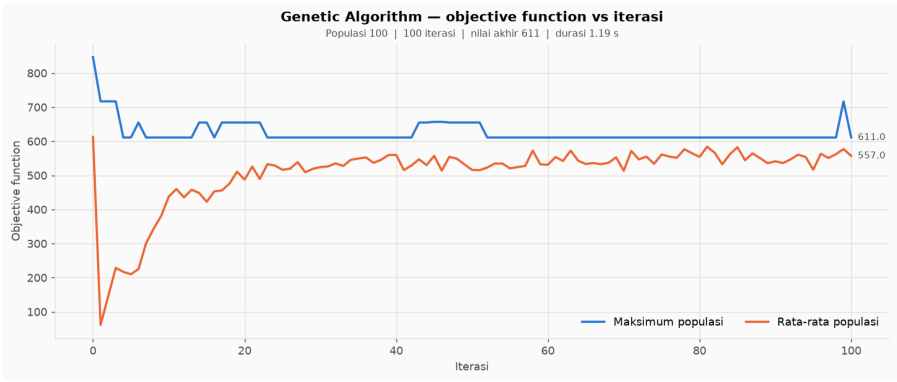
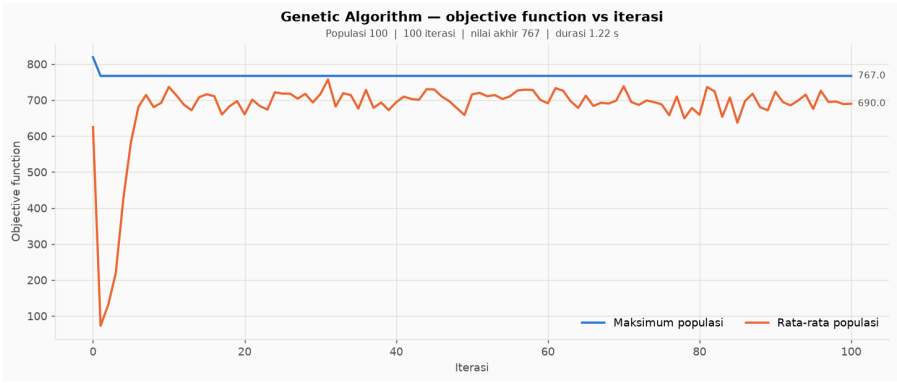
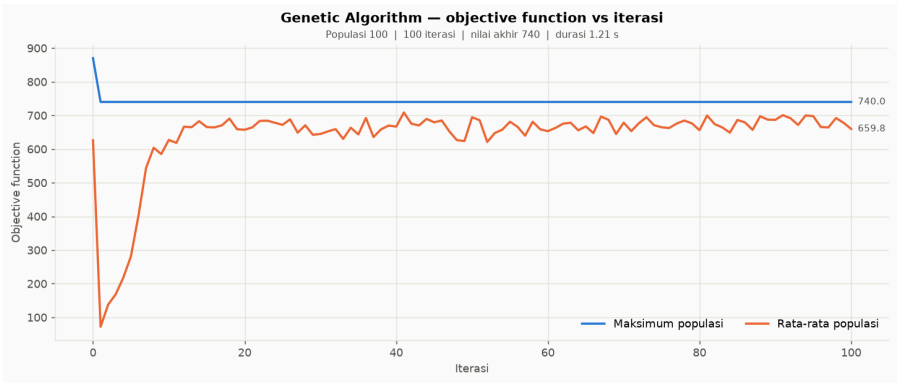
3	

Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.68	0.60	0.57
Jumlah Iterasi	100	100	100
Initial State	869	826	826
Final State	641	634	777

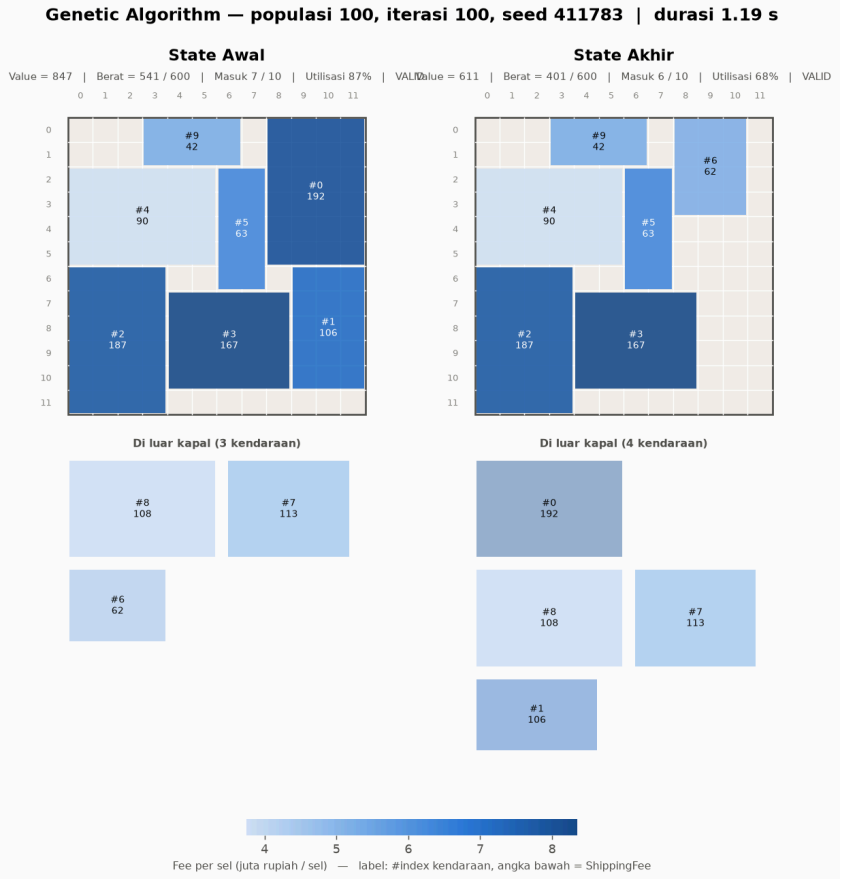
.

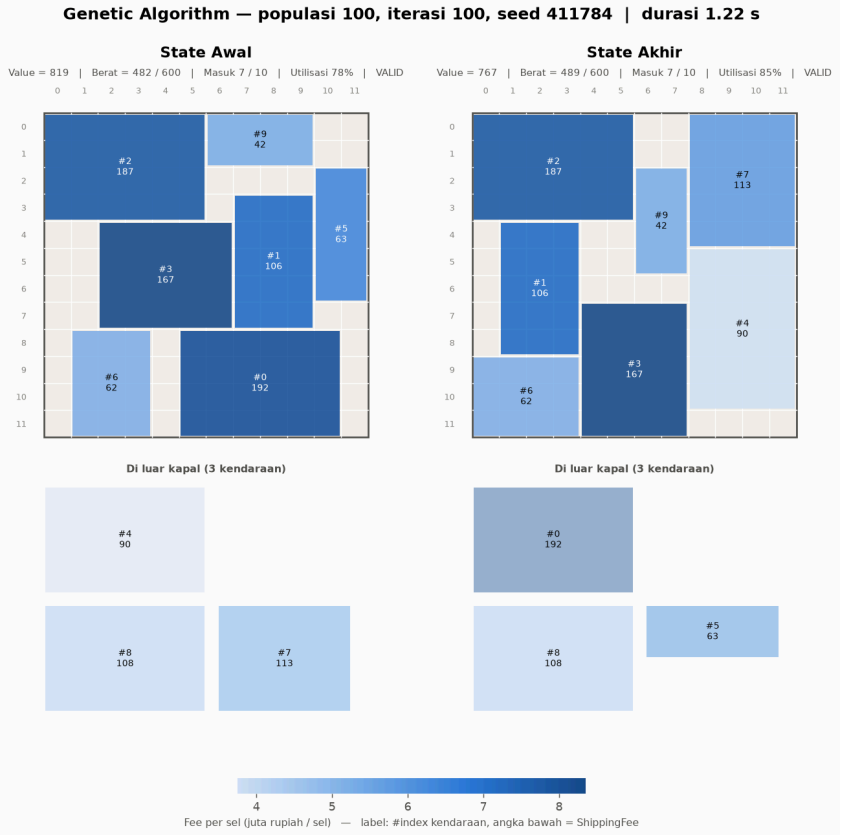
c) Populasi 100, Iterasi 100

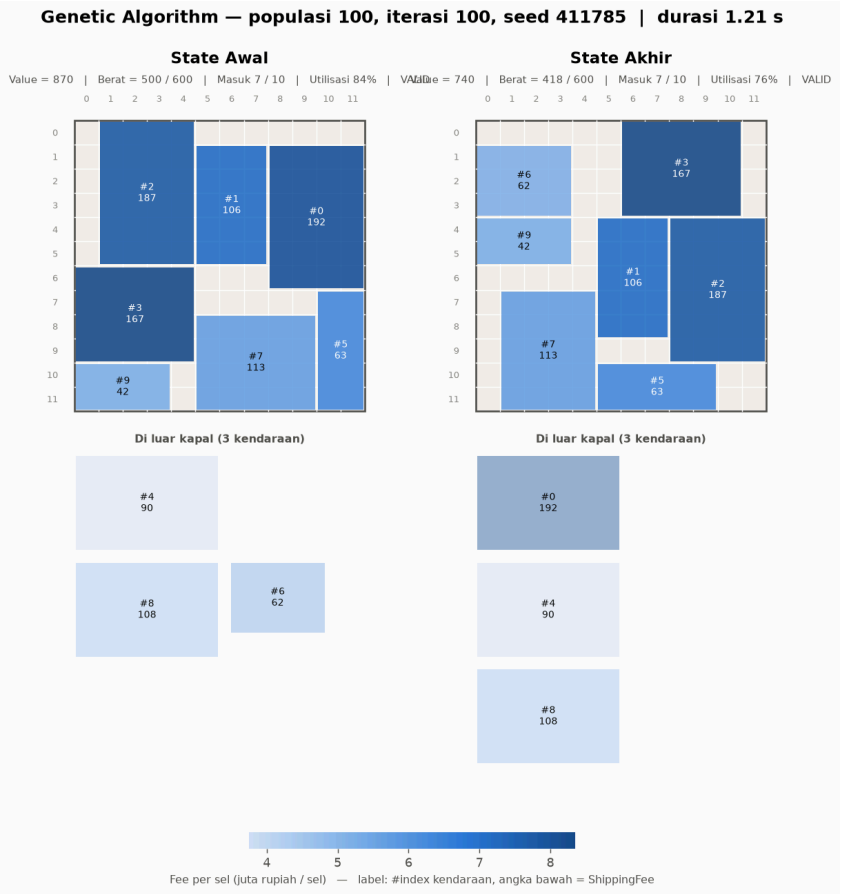
Percobaan	**Grafik Nilai Objective Function terhadap Banyak Iterasi**

1	
2	
3	

Percobaan	Perbandingan State

1	 Genetic Algorithm — populasi 100, iterasi 100, seed 411783 | durasi 1.19 s State Awal Value = 847 | Berat = 541 / 600 | Masuk 7 / 10 | Utilisasi 87% | VALID #9 42 #0 192 #4 90 #5 63 #2 187 #3 167 #1 106 State Akhir Value = 611 | Berat = 401 / 600 | Masuk 6 / 10 | Utilisasi 68% | VALID #9 42 #6 62 #4 90 #5 63 #2 187 #3 167 Di luar kapal (3 kendaraan) #8 108 #7 113 #6 62 Di luar kapal (4 kendaraan) #0 192 #8 108 #7 113 #1 106 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

2	 Genetic Algorithm — populasi 100, iterasi 100, seed 411784 | durasi 1.22 s State Awal Value = 819 | Berat = 482 / 600 | Masuk 7 / 10 | Utilisasi 78% | VALID State Akhir Value = 767 | Berat = 489 / 600 | Masuk 7 / 10 | Utilisasi 85% | VALID #2 187 #9 42 #5 63 #3 167 #1 106 #6 62 #0 192 #7 113 #4 90 Di luar kapal (3 kendaraan) #8 108 4 5 6 7 8 Fee per sel (juta rupiah / sel) — label: #index kendaraan, angka bawah = ShippingFee

3	

Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	1.19	1.22	1.21
Jumlah Iterasi	100	100	100
Initial State	847	819	870
Final State	611	767	740

Analisis Variasi Populasi

Pada variasi ini banyak iterasi ditetapkan 100, sedangkan jumlah populasi divariasikan menjadi 20, 50, dan 100. Ringkasan hasilnya ditunjukkan pada tabel berikut.

Konfigurasi	**Nilai awal terbaik**	**Nilai akhir**	**Rata-rata akhir**	**Rata-rata durasi (s)**

Populasi 20	745, 819, 779	745, 531, 742	672,7	0,25
Populasi 50	869, 826, 826	641, 634, 777	684,0	0,62
Populasi 100	847, 819, 870	611, 767, 740	706,0	1,21

Tabel ringkasan variasi populasi Genetic Algorithm (100 iterasi)

Berbeda dengan variasi iterasi, menambah jumlah populasi sedikit memperbaiki rata-rata nilai akhir, dari 672,7 (populasi 20) menjadi 684,0 (populasi 50) dan 706,0 (populasi 100). Populasi yang besar menyimpan lebih banyak individu valid dan lebih beragam, sehingga peluang terbentuknya anak yang baik lebih besar dan individu bagus tidak mudah hilang seluruhnya. Sebaliknya, populasi 20 memiliki keragaman rendah dan hasilnya paling tidak stabil: percobaan 1 mempertahankan nilai awalnya (745), sedangkan percobaan 2 turun 288 menjadi 531, nilai akhir terendah dari seluruh eksperimen Genetic Algorithm.

Konfigurasi populasi 50 dengan 100 iterasi sama dengan konfigurasi pada variasi iterasi, sehingga nilai awal dan akhirnya identik. Hal ini menunjukkan bahwa dengan seed yang sama, program menghasilkan proses pencarian yang sama dan dapat direproduksi; yang berbeda hanya durasi karena kondisi komputer saat dijalankan. Durasi naik hampir linear terhadap jumlah populasi (0,25s, 0,62s, dan 1,21s), karena jumlah evaluasi fitness per generasi sama dengan jumlah populasi. Meskipun hasilnya membaik, peningkatan kualitas (sekitar 33 dari populasi 20 ke 100) tidak sebanding dengan durasi yang hampir lima kali lipat.

Analisis Keseluruhan Genetic Algorithm

Dari 18 percobaan Genetic Algorithm, rata-rata nilai akhir adalah 694,3 dengan nilai akhir terbaik 847. Sebanyak 16 percobaan berakhir dengan nilai yang lebih rendah daripada individu terbaik populasi awal, dua percobaan sama dengan nilai awal, dan tidak ada satu pun percobaan yang melampaui nilai awal. Padahal nilai awal terbaik populasi cukup tinggi (745–870), karena dari 20–100 individu acak biasanya sudah ada beberapa state yang baik. Dengan kata lain, pada data uji ini Genetic Algorithm lebih sering kehilangan solusi baik daripada menemukan solusi yang lebih baik.

Penyebab utamanya ada dua. Pertama, kromosom berisi posisi dan orientasi setiap kendaraan, sehingga one-point crossover menggabungkan potongan dua induk yang susunannya tidak saling cocok. Anak yang dihasilkan sering bertumpuk atau melebihi kapasitas dan bernilai

0, terlihat dari rata-rata populasi yang anjlok pada generasi pertama. Kedua, sesuai pseudocode PPT, populasi baru seluruhnya terdiri dari anak tanpa elitisme, sehingga individu terbaik tidak dijamin bertahan. Faktor iterasi memperburuk hasil, sedangkan faktor populasi sedikit memperbaikinya.

Dibandingkan algoritma lain, nilai akhir Genetic Algorithm (rata-rata 694,3) berada di bawah Sideways Move (933,3), Random Restart (890,7), Simulated Annealing (822,0), dan Steepest Ascent (736,0), serta setara dengan Stochastic Hill-Climbing (701,0). Durasinya (0,25–1,21s) juga termasuk yang paling lama.

KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil eksperimen pada bagian Hasil dan Analisis, ringkasan setiap algoritma ditunjukkan pada tabel berikut.

Algoritma	Nilai akhir per percobaan	Rata-rata	Terbaik	Rata-rata durasi (s)
Steepest Ascent HC	632, 757, 819	736,0	819	0,02
HC with Sideways Move	932, 936, 932	933,3	936	0,27
Random Restart HC	870, 932, 870	890,7	932	0,25
Stochastic HC	819, 632, 652	701,0	819	0,76
Simulated Annealing	828, 819, 819	822,0	828	0,07
GA (pop 50, iter 50 / 100 / 200)	rata-rata 781,0 / 684,0 / 638,0	701,0	847	0,51 – 0,79
GA (iter 100, pop 20 / 50 / 100)	rata-rata 672,7 / 684,0 / 706,0	687,6	777	0,25 – 1,21

Tabel ringkasan hasil eksperimen (nilai objective function = total ShippingFee, juta rupiah)

Dari hasil tersebut, dapat ditarik beberapa kesimpulan berikut.

1. Nilai tertinggi yang mungkin adalah 936. Nilai ini benar-benar dicapai oleh Hill-Climbing with Sideways Move pada percobaan kedua, sehingga 936 merupakan optimum global data uji. Sideways Move adalah algoritma yang paling dekat dengan optimum global (rata-rata 933,3, selisih paling banyak 4), diikuti Random Restart (890,7), Simulated Annealing (822,0), Steepest Ascent (736,0), dan Stochastic Hill-Climbing (701,0). Genetic Algorithm menghasilkan nilai akhir terendah (rata-rata 638,0 sampai 781,0 per konfigurasi).

2. Nilai objective function hanya bergantung pada kendaraan mana yang dimuat, bukan pada posisinya, sehingga banyak neighbor memiliki nilai yang sama dan ruang pencarian memiliki banyak dataran (shoulder atau flat). Steepest Ascent berhenti setelah 2–3 iterasi karena tidak ada neighbor yang lebih baik, sedangkan Sideways Move dapat terus berpindah di dataran tersebut sampai menemukan jalan naik. Random Restart memperbaiki Steepest Ascent dengan mencoba beberapa state awal dan mengambil yang terbaik. Stochastic Hill-Climbing hanya mencoba satu successor acak per iterasi, sehingga 1000 iterasi belum cukup untuk naik jauh dari state awal. Simulated Annealing memberikan hasil menengah karena dengan T0 = 20 dan faktor pendinginan 0,995 suhu turun cukup cepat, sehingga setelah beberapa ratus iterasi perilakunya mendekati Stochastic Hill-Climbing dan berhenti di optimum lokal.
3. Genetic Algorithm kurang cocok untuk representasi ini**.** One-point crossover menggabungkan posisi kendaraan dari dua induk yang susunannya berbeda, sehingga anak yang dihasilkan sering bertumpuk atau melebihi kapasitas dan bernilai 0. Karena seluruh populasi diganti setiap generasi tanpa elitisme, individu terbaik dapat hilang, sehingga nilai akhir sering lebih rendah daripada individu terbaik populasi awal. Semakin banyak iterasi, semakin besar peluang kehilangan individu terbaik: rata-rata nilai akhir turun dari 781,0 (50 iterasi) menjadi 684,0 (100 iterasi) dan 638,0 (200 iterasi). Sebaliknya, populasi yang lebih besar sedikit memperbaiki hasil (672,7; 684,0; 706,0 untuk populasi 20, 50, dan 100) karena lebih banyak individu valid yang dapat bertahan.
4. Durasi Simulated Annealing (0,07s) dan Steepest Ascent (0,02s) adalah yang tercepat. Sideways Move (0,27s) dan Random Restart (0,25s) lebih lama karena melakukan lebih banyak iterasi atau beberapa kali pencarian. Stochastic Hill-Climbing (0,76s) relatif lambat karena setiap iterasi membangkitkan daftar seluruh aksi sebelum memilih satu secara acak. Durasi Genetic Algorithm (0,25–1,21s) bertambah sebanding dengan jumlah populasi dikali banyak iterasi.
5. Secara konsistensi, Sideways Move paling konsisten (932–936), disusul Simulated Annealing (819–828) dan Random Restart (870–932). Steepest Ascent (632–819) dan Stochastic Hill-Climbing (632–819) sangat bergantung pada state awal, sedangkan Genetic Algorithm memiliki variasi yang besar antara percobaan (misalnya 531–745 pada populasi 20).

6. Secara keseluruhan, untuk kasus ini **Hill-Climbing with Sideways Move** memberikan kombinasi terbaik antara kualitas solusi, konsistensi, dan waktu komputasi.

B. Saran

1. **Genetic Algorithm:** menambahkan elitisme (mempertahankan beberapa individu terbaik) dan/atau heuristik repair yang memperbaiki anak yang tidak valid, atau merancang crossover yang menjaga validitas state. Kedua heuristik ini sudah tersedia sebagai opsi tambahan di program dan dapat dibandingkan dengan versi sesuai PPT.
2. **Simulated Annealing:** menggunakan jadwal pendinginan yang lebih lambat (misalnya faktor 0,999) atau menyesuaikan T0 dengan skala selisih nilai objective function (puluhan sampai ratusan juta rupiah), sehingga fase eksplorasi berlangsung lebih lama sebelum suhu mendekati nol.
3. Agar lebih banyak iterasi dapat dijalankan dalam waktu yang sama pada stochastic hill-climbing memperbesar nmax dan membangkitkan successor acak secara langsung tanpa membuat daftar seluruh aksi.
4. Menggunakan data uji yang lebih "menantang", misalnya total luas kendaraan melebihi luas geladak dan dengan beberapa ukuran kapal, serta menambah jumlah percobaan lebih dari tiga kali agar perbandingan antara algoritma lebih jelas dan kesimpulan lebih kuat secara statistik.
5. Merancang objective function lain yang juga memperhitungkan ETA, misalnya memberi prioritas pada kendaraan dengan deadline terdekat, sesuai spesifikasi bonus.

.

REFERENSI

Dafda, “Simulated Annealing search with real example in Artificial Intelligence & its Python implementation”, Youtube, 2026 [Video]. Available: Simulated Annealing [Accessed: 20 October 2026]

Slide Kuliah IF3070 Dasar Inteligensi Artifisial 2026, “Beyond Classical Search”. Edunex, 2026, [PPT]. [Accessed: 12 October 2026]

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	: Muhammad Azka Firdaus/18224036 Almer Zain Farriseno/18224070 Fatin Sajid Darwis/18224074
Fakultas/Sekolah	: STEI-K
Kode Mata Kuliah	: IF3070
Nama Mata Kuliah	: Dasar Intelegensi Artifisial
Judul Tugas/Proposal	: Tugas Besar 1 Dasar Intelegensi Artifisial

Menyatakan bahwa kami menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026







Almer Zain Farisseno	Fatin Sajid Darwis	Muhammad Azka Firdaus
18224070	18224074	18224036